



## ***Opus Codec*** **Programmer's Guide**

For HiFi DSPs



Cadence Design Systems, Inc.  
2655 Seely Ave.  
San Jose, CA 95134  
[www.cadence.com](http://www.cadence.com)

Copyright © 2024 Cadence Design Systems, Inc.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

**Trademarks:** Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

**Restricted Print Permission:** This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

- The publication may be used solely for personal, informational, and noncommercial purposes;
- The publication may not be modified in any way;
- Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement;
- The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration; and
- Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

**Disclaimer:** Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence's customer in accordance with, a written agreement between Cadence and its customer. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

**Restricted Rights:** Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

For further assistance, contact Cadence Online Support at <https://support.cadence.com/>.

**Version:** 1.12

**Last Updated:** September 2024

Cadence Design Systems, Inc.  
2655 Seely Ave.  
San Jose, CA 95134  
[www.cadence.com](http://www.cadence.com)

# Contents

|                                                |     |
|------------------------------------------------|-----|
| Contents .....                                 | iii |
| Figures .....                                  | iv  |
| Tables .....                                   | v   |
| Document Change History .....                  | vi  |
| 1. Introduction to the HiFi Opus Codec .....   | 1   |
| 1.1 Opus Codec Description .....               | 1   |
| 1.2 Document Overview .....                    | 1   |
| 1.3 HiFi Opus Codec Specifications .....       | 2   |
| 1.4 HiFi Codec Performance .....               | 3   |
| 1.4.1 Memory .....                             | 4   |
| 1.4.2 Timings .....                            | 5   |
| 2. Generic HiFi Speech Codec API .....         | 7   |
| 2.1 Memory Management .....                    | 7   |
| 2.1.1 API Handle/Persistent Memory .....       | 7   |
| 2.1.2 Scratch Memory .....                     | 8   |
| 2.1.3 Input Buffer .....                       | 8   |
| 2.1.4 Output Buffer .....                      | 8   |
| 2.2 C Language APIs .....                      | 8   |
| 2.3 About Generic API Errors .....             | 9   |
| 2.4 Files Supporting the APIs .....            | 10  |
| 3. HiFi DSP Opus Codec API .....               | 11  |
| 3.1 Opus Codec-specific Files .....            | 11  |
| 3.2 I/O Formats .....                          | 11  |
| 3.3 Control Structure for Opus Codec .....     | 12  |
| 3.4 API Functions .....                        | 16  |
| 3.4.1 Startup Stage .....                      | 16  |
| 3.4.2 Memory Allocation Stage .....            | 17  |
| 3.4.3 Initialization Stage .....               | 18  |
| 3.4.4 Process Stage .....                      | 22  |
| 4. Introduction to the Example Testbench ..... | 29  |
| 4.1 Making Executables .....                   | 29  |
| Building OPUS Applications .....               | 29  |
| 4.2 Usage .....                                | 32  |
| 4.2.1 OPUS Usage .....                         | 32  |
| 4.2.2 OPUS-SILK Usage .....                    | 36  |

|       |                                                                      |    |
|-------|----------------------------------------------------------------------|----|
| 4.2.3 | OPUS-CELT Usage .....                                                | 39 |
| 5.    | Appendix – Ogg Container Format Support in Encoder and Decoder ..... | 42 |
| 5.1   | Ogg Packer/Parser-specific Files.....                                | 42 |
| 5.2   | I/O Formats .....                                                    | 42 |
| 5.3   | Configuration Structure .....                                        | 43 |
| 5.4   | API Functions .....                                                  | 43 |
| 5.4.1 | Memory Allocation Stage .....                                        | 43 |
| 5.4.2 | Initialization Stage .....                                           | 45 |
| 5.4.3 | Process Stage .....                                                  | 47 |
| 5.4.4 | HiFi Ogg Packer/Parser Parameters .....                              | 55 |
| 5.5   | Metadata for Non-Ogg/Raw Decoder .....                               | 56 |
| 6.    | References .....                                                     | 57 |

## Figures ---

|          |                                    |   |
|----------|------------------------------------|---|
| Figure 1 | HiFi Speech Codec Interfaces ..... | 7 |
| Figure 2 | Speech Codec Flow Overview.....    | 9 |

## Tables

---

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Table 1-1 Opus Codec Configurations.....                                   | 3  |
| Table 1-2 Library Memory .....                                             | 4  |
| Table 1-3 Run-time Memory .....                                            | 4  |
| Table 1-4 Encoder Timings .....                                            | 5  |
| Table 1-5 Decoder Timings.....                                             | 5  |
| Table 1-6 Ogg Parser Timings.....                                          | 6  |
| Table 1-7 Ogg Packer Timings .....                                         | 6  |
| Table 2-1 Error Codes Format .....                                         | 9  |
| Table 3-1 Encoder Control Structure xa_opus_enc_control_t Parameters ..... | 12 |
| Table 3-2 Decoder Control Structure xa_opus_dec_control_t Parameters.....  | 15 |
| Table 3-3 Library Identification Functions .....                           | 16 |
| Table 3-4 Memory Management Functions .....                                | 17 |
| Table 3-5 Opus Encoder's Initialization Function .....                     | 18 |
| Table 3-6 Opus Decoder's Initialization Function .....                     | 20 |
| Table 3-7 Macros to Calculate I/O Buffer Sizes and Configuration .....     | 22 |
| Table 3-8 Opus Encoder's Process Function .....                            | 23 |
| Table 3-9 Opus Decoder's Process Function .....                            | 26 |
| Table 5-1 xa_ogg_pack_init_cfg_t Structure Parameters.....                 | 43 |
| Table 5-2 xa_ogg_parse_init_cfg_t Structure Parameters .....               | 43 |
| Table 5-3 Memory Management Functions .....                                | 44 |
| Table 5-4 HiFi Ogg Pack Initialization Function Details .....              | 45 |
| Table 5-5 HiFi Ogg Parse Initialization Function Details .....             | 46 |
| Table 5-6 Process Stage Functions.....                                     | 47 |
| Table 5-7 HiFi Ogg Packer Process Function Details .....                   | 47 |
| Table 5-8 HiFi Ogg Parser Process Function Details .....                   | 49 |
| Table 5-9 HiFi Ogg Packer Set Parameter Function Details .....             | 51 |
| Table 5-10 HiFi Ogg Parser Set Parameter Function Details.....             | 52 |
| Table 5-11 HiFi Ogg Packer Get Parameter Function Details.....             | 53 |
| Table 5-12 HiFi Ogg Parser Get Parameter Function Details .....            | 54 |
| Table 5-13 HiFi Ogg Packer Parameters.....                                 | 55 |
| Table 5-14 HiFi Ogg Parser Parameters .....                                | 55 |

## Document Change History

| Version | Changes                                                                                                                                                                                                                                                                                                                                                                                |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0     | <ul style="list-style-type: none"> <li>■ Initial revision</li> </ul>                                                                                                                                                                                                                                                                                                                   |
| 1.1     | <ul style="list-style-type: none"> <li>■ Updated the performance data in Section 1.4</li> </ul>                                                                                                                                                                                                                                                                                        |
| 1.2     | <ul style="list-style-type: none"> <li>■ Updated performance data in Section 1.4.</li> <li>■ Added two new error codes in Section 3.4.3:               <ul style="list-style-type: none"> <li>■ XA_OPUS_CONFIG_FATAL_ENC_INVALID_PARAM_COMBINATION</li> <li>■ XA_OPUS_EXECUTE_NONFATAL_INVALID_FORCED_MODE_SETTINGS</li> </ul> </li> </ul>                                             |
| 1.2.1   | <ul style="list-style-type: none"> <li>■ Reference source code version updated to v1.1.3.</li> <li>■ Updated performance data in Section 1.4.</li> <li>■ Added control parameter 'variable_duration' for encoder in Section 3.3.</li> </ul>                                                                                                                                            |
| 1.3     | <ul style="list-style-type: none"> <li>■ Added Ogg container support for Opus Decoder in Section 5.</li> <li>■ Added performance data for Ogg container support in Section 1.4.</li> <li>■ Added six new control parameters for decoder in Section 3.3.</li> </ul>                                                                                                                     |
| 1.4     | <ul style="list-style-type: none"> <li>■ Reference source code version updated to v1.2.1.</li> <li>■ Control parameter 'variable_duration' usage is deprecated. Updated Section 1.3, 3.3, 3.4.4, and 0 accordingly.</li> <li>■ Updated performance data in Section 1.4.</li> <li>■ Control parameter 'framesize' supports additional values. Updated Section 0 accordingly.</li> </ul> |
| 1.5     | <ul style="list-style-type: none"> <li>■ Added performance data for Fusion F1 DSP in Section 1.41.</li> </ul>                                                                                                                                                                                                                                                                          |

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.6  | <ul style="list-style-type: none"> <li>■ Updated performance data, added new multistream decoder case.</li> <li>■ Added three new control parameters in decoder control structure to support multistream decoding in Section 3.3.</li> <li>■ Error code <code>XA_OPUS_CONFIG_FATAL_ENC_INVALID_PARAM_COMBINATION</code> is replaced by <code>XA_OPUS_CONFIG_FATAL_INVALID_PARAM_COMBINATION</code> for <code>xa_opus_enc_init</code> in Section 3.4.3.</li> <li>■ Error code <code>XA_OPUS_CONFIG_FATAL_DEC_MULTISTREAM_DECODE_NOT_SUPPORTED</code> is removed and <code>XA_OPUS_CONFIG_FATAL_INVALID_PARAM_COMBINATION</code>, <code>XA_OPUS_CONFIG_FATAL_DEC_NUM_STREAMS_NOT_SUPPORTED</code>, <code>XA_OPUS_CONFIG_FATAL_DEC_UNKNOWN_CHANNEL_MAPPING</code>, <code>XA_OPUS_CONFIG_FATAL_DEC_BAD_STREAM_MAP</code>, <code>XA_OPUS_CONFIG_NONFATAL_DEC_STREAM_MAP_IGNORED</code> added for <code>xa_opus_dec_init</code> in Section 3.4.3.</li> <li>■ Added new macro <code>XA_OPUS_MAX_NUM_STREAMS</code> and updated value of <code>XA_OPUS_MAX_NUM_CHANNELS</code> from 2 to 8 in Section 3.4.4.</li> <li>■ Updated decoder testbench help in Section 4.2, <code>-numch</code> range is now 0-8 and new option <code>-strmap</code> is added.</li> </ul> |
| 1.7  | <ul style="list-style-type: none"> <li>■ Added performance data for HiFi 5 in Section 1.4.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 1.8  | <ul style="list-style-type: none"> <li>■ Reference source code version updated to v1.3.1.</li> <li>■ Updated performance data in Section 1.4.</li> <li>■ Now released in three packages: codec, decoder, and encoder.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 1.9  | <ul style="list-style-type: none"> <li>■ Now released in single package that contains three libraries (codec, decoder, and encoder). Added instructions for switching library in Section 4.1.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 1.10 | <ul style="list-style-type: none"> <li>■ Updated performance data for all cores and added new column for HiFi1 core in Section 1.4.</li> <li>■ Updated Section 4 to list testbench error handling source files.</li> <li>■ Section 4.2 updated to add details about constraint on sweep.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 1.11 | <ul style="list-style-type: none"> <li>■ Updated build instructions in Section 4.1. Added support for standalone OPUS-SILK and OPUS-CELT libraries for encoder as well as decoder.</li> <li>■ Added information about usage of OPUS-SILK and OPUS-CELT libraries in Sections 4.2.2 and 4.2.3.</li> <li>■ Updated Memory requirements in Section 1.4.1. Added support for config based dynamic memory allocation.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      | <ul style="list-style-type: none"><li>■ Added the following error codes in Section 3.4.4:<ul style="list-style-type: none"><li>■ XA_OPUS_EXECUTE_NONFATAL_UNSUPPORTED_MODE</li><li>■ XA_OPUS_EXECUTE_NONFATAL_HYBRID_MODE_DECODED</li></ul></li></ul>                                                                                                                                                                                                                                                                                                  |
| 1.12 | <ul style="list-style-type: none"><li>■ Added Multichannel encoder support.</li><li>■ Added six new control parameters for the encoder in Section 3.3.</li><li>■ Updated the details of the new encoder testbench parameters required for Ogg support in Section 4.2.</li><li>■ Updated the details of testbench parameters required to support the new channel mapping family 255 in Section 4.2.</li><li>■ Added Ogg container support for the Opus Encoder in Section 5.</li><li>■ Updated performance and memory numbers in Section 1.4.</li></ul> |

---

# 1. Introduction to the HiFi Opus Codec

---

The HiFi DSP Opus Codec implements the speech and audio codec standard specified in the definition of the Opus audio codec (RFC 6716) <sup>[1]</sup>. It also supports encoding/decoding of Opus streams in Ogg container format.

The vendor source version is libopus 1.3.1 <sup>[2]</sup>.

## 1.1 Opus Codec Description

The Opus codec is designed to handle a wide range of interactive audio applications, including Voice over IP, video conferencing, in-game chat, and even live distributed music performances. It scales from low-bitrate narrowband speech at 6 kbps to very high-quality stereo music at 510 kbps. Opus uses both Linear Prediction (LP) and the Modified Discrete Cosine Transform (MDCT) to achieve good compression of both speech and music.

## 1.2 Document Overview

This document covers all the information required to integrate the HiFi speech codecs into an application. The HiFi codec libraries implement a simple API to encapsulate the complexities of the coding operations and simplify the application and system implementation.

This document consists of the following sections:

- Section 1 describes the HiFi Opus in brief.
- Section 2 describes parts of API that are common to all the HiFi codecs.
- Section 3 describes all the features and information particular to the HiFi Opus Codec.
- Section 4 describes an example testbench.
- Section 5 Ogg container format support interface of the HiFi Opus Codec.
- Section 6 lists the references.

## 1.3 HiFi Opus Codec Specifications

The HiFi DSP Opus Codec implements the following features:

- Uses the Cadence Speech Codec API
- Encoder input/decoder output PCM: 16 bits per sample
- Supported input sampling frequencies: 8 kHz, 12 kHz, 16 kHz, 24 kHz, 48 kHz
- Support the following complexity modes: voip, audio, and restricted-lowdelay
- Supports five audio bandwidths:
  - Narrowband (NB): 8 kHz effective sampling rate (4 kHz audio bandwidth)
  - Mediumband (MB): 12 kHz effective sampling rate (6 kHz audio bandwidth)
  - Wideband (WB): 16 kHz effective sampling rate (8 kHz audio bandwidth)
  - Super wideband (SWB): 24 kHz effective sampling rate (12 kHz audio bandwidth)
  - Full-band (FB): 48 kHz effective sampling rate (20 kHz audio bandwidth)
- Constant and variable bitrate encoding from 6 kbps to 510 kbps<sup>1</sup>
- Discontinuous Transmission (DTX)
- Voice Activity Detection (VAD)
- Three operating modes of OPUS libraries:
  - SILK-only mode: for use in low-bitrate connections with an audio bandwidth of WB or less
  - Hybrid (SILK+CELT) mode: for SWB or FB speech at medium bitrates
  - CELT-only mode: for a very low delay speech transmission and music transmission (NB to FB)
- Supports Forward Error Correction (FEC)
- Packet Loss Resilience
- Frame duration: 2.5, 5, 10, 20, 40, 60, 80, 100, and 120 milliseconds(ms)
- Supports the Ogg container format in decoder and encoder.
- Supports the standalone OPUS-SILK and OPUS-CELT encoder and decoder libraries
- Stream/Config parameter-based memory allocation

---

<sup>1</sup> RFC6716 Section 2.1.1

Table 1-1 shows the possible configurations of operating mode, audio bandwidth, and frame size.

Table 1-1 Opus Codec Configurations

| Mode      | Bandwidth | Frame sizes in ms                    |
|-----------|-----------|--------------------------------------|
| SILK-only | NB        | 10, 20, 40, 60, 80, 100, 120         |
| SILK-only | MB        | 10, 20, 40, 60, 80, 100, 120         |
| SILK-only | WB        | 10, 20, 40, 60, 80, 100, 120         |
| Hybrid    | SWB       | 10, 20                               |
| Hybrid    | FB        | 10, 20                               |
| CELT-only | NB        | 2.5, 5, 10, 20, 40, 60, 80, 100, 120 |
| CELT-only | WB        | 2.5, 5, 10, 20, 40, 60, 80, 100, 120 |
| CELT-only | SWB       | 2.5, 5, 10, 20, 40, 60, 80, 100, 120 |
| CELT-only | FB        | 2.5, 5, 10, 20, 40, 60, 80, 100, 120 |

The decoder implementation is verified to be bit-exact for all the test vectors available with the libopus 1.3.1 package. The encoder implementation is verified for internally generated test vectors and matches bit-exact with the reference implementation.

As some specific applications require only one of the SILK-only or CELT-only operating modes, standalone libraries for OPUS-SILK and OPUS-CELT encoder/decoder are provided. This reduces static and dynamic memory requirements by removing overheads of unused operating modes. For more information on the memory requirements of the standalone libraries, see Section 1.4.1.

---

**Note** The OPUS-SILK and OPUS-CELT libraries follow Generic HiFi Speech Codec API and HiFi DSP Opus Codec API as described in Section 2 and Section 3. The encoder/decoder control parameters that are not supported in standalone libraries are mentioned in the description (see Section 3.3).

---

For the OPUS SILK/CELT standalone decoder binary, handling the Hybrid mode frames is challenging. The OPUS-SILK decoder is equipped with a feature to decode hybrid-mode frames with limited quality. The CELT mode is considered invalid for the OPUS-SILK standalone decoder. If the CELT mode is detected, such a frame is skipped, and decoding is resumed for the next valid frame. Similarly, SILK mode and Hybrid mode frames are considered invalid for the OPUS-CELT decoder and are skipped for decoding, and a Non-Fatal error is returned. Warnings/Non-Fatal errors used for this feature are listed in Section 3.4.4 (Process stage of the decoder). This feature ensures that the OPUS-SILK/OPUS-CELT decoder is compatible with any Opus encoder and can handle opus-encoded frames using different types of modes.

The Opus standard reference C code allocates memories assuming the worst-case scenario, which results in higher memory requirements for some scenarios. The Opus codec is enhanced with a feature to decide the dynamic memory requirement based on the stream and config parameters provided by the application. This feature reduces persistent/scratch/input/output memory footprint. It is achieved by calculating memory requirements based on config parameters like sample rate, number of channels, and so on, instead of allocating memory based on the maximum possible value for the given config parameter.

## 1.4 HiFi Codec Performance

The HiFi DSP Opus codec is characterized on the 5-stage HiFi DSP. The memory usage and performance figures are provided for design reference.

Opus is more efficient when operating with variable bitrate (VBR) mode (default mode). Compared to the VBR mode, the constant bitrate (CBR) mode has lower audio quality at a given average bitrate and has higher computational complexity. For very low bitrates, the average CPU load (MHz) for the CBR mode is approximately 2.5 to 3 times that of the VBR mode.

## 1.4.1 Memory

Table 1-2 Library Memory

| Library              | Code (Kbytes) |        |         |        |        | Data (Kbytes) |
|----------------------|---------------|--------|---------|--------|--------|---------------|
|                      | HiFi 1        | HiFi 3 | HiFi 3z | HiFi 4 | HiFi 5 |               |
| xa_opus_codec.a      | 199.23        | 201.75 | 212.12  | 223.70 | 232.40 | 25.20         |
| xa_opus_dec.a        | 81.90         | 83.08  | 89.76   | 94.11  | 100.65 | 22.42         |
| xa_opus_enc.a        | 158.64        | 161.25 | 169.45  | 179.63 | 187.10 | 25.03         |
| xa_opus_silk_codec.a | 127.0         | 125.25 | 133.44  | 139.50 | 145.86 | 9.29          |
| xa_opus_silk_enc.a   | 103.28        | 101.89 | 108.72  | 113.57 | 120.22 | 9.21          |
| xa_opus_silk_dec.a   | 44.85         | 44.42  | 48.78   | 50.91  | 54.44  | 7.40          |
| xa_opus_celt_codec.a | 113.47        | 116.60 | 123.62  | 131.0  | 135.43 | 17.59         |
| xa_opus_celt_enc.a   | 88.57         | 91.68  | 97.17   | 103.61 | 106.91 | 17.44         |
| xa_opus_celt_dec.a   | 57.28         | 58.08  | 63.42   | 66.44  | 71.66  | 16.20         |

## Encoder and Decoder

Table 1-3 Run-time Memory

|                                                                    | Run-Time Memory (Kbytes) |         |       |       |        |
|--------------------------------------------------------------------|--------------------------|---------|-------|-------|--------|
|                                                                    | Persistent               | Scratch | Stack | Input | Output |
| Opus Encoder (2 channels, 48KHz)                                   | 29.0                     | 75.5    | 11.8  | 3.7   | 1.5    |
| Opus Encoder (8 channels, 5 streams – 3 coupled, 2 uncoupled)      | 121.1                    | 109     | 11.8  | 15    | 6.3    |
| Opus Celt Encoder (2 channels, 48KHz)                              | 12.0                     | 71.5    | 11.8  | 3.7   | 1.5    |
| Opus Encoder (1 Channel, 16KHz)                                    | 15.9                     | 50.5    | 11.0  | 0.6   | 1.5    |
| Opus Silk Encoder (1 Channel, 16KHz)                               | 11.1                     | 50.5    | 10.9  | 0.6   | 1.5    |
| Opus Decoder Stereo (48KHz)                                        | 26.3                     | 12.0    | 2.8   | 2.9   | 22.5   |
| Opus Celt Decoder -Stereo (48KHz)                                  | 18.0                     | 12.0    | 2.7   | 2.9   | 22.5   |
| Opus Decoder (6 channels, 4 streams - 2 coupled, 2 uncoupled)      | 87.3                     | 34.5    | 2.8   | 70.5  | 67.5   |
| Opus Celt Decoder (6 channels, 4 streams - 2 coupled, 2 uncoupled) | 53.9                     | 34.5    | 2.7   | 70.5  | 67.5   |
| Ogg parser (1 coupled stream)                                      | 26.3                     | 12      | 0.6   | 4.0   | 2.9    |
| Ogg parser (4 streams – 2 coupled, 2 uncoupled)                    | 87.32                    | 34.5    | 0.6   | 4.0   | 70.5   |
| Ogg packer (5 streams – 3 coupled, 2 uncoupled)                    | 76.5                     | 0       | 0.3   | 6.3   | 64.0   |

**Note** The Ogg parser's runtime memory is only required for the Ogg Opus streams.

**Note** The Ogg parser's persistent memory requirement is dependent on the maximum page size allowed and is configurable. The size reported above is for a default page size of 64 KB.

**Note** For the Ogg Parser, the input buffer size does not have any restrictions. If the input buffer is too small, it may require multiple calls to the Ogg Parser to get an Ogg page.  
For the Ogg Parser, the output buffer size does not have any restrictions. However, it should be large enough to hold at least one Opus packet. If the comment packet is bigger than the Opus packets, then the output buffer should be large enough to hold the comment packet.

## 1.4.2 Timings

### Encoder

Table 1-4 Encoder Timings

| Rate<br>kHz | Nch | Bitrate<br>(kbps) | Average CPU Load (MHz) |        |         |        |        |                         |
|-------------|-----|-------------------|------------------------|--------|---------|--------|--------|-------------------------|
|             |     |                   | HiFi 1                 | HiFi 3 | HiFi 3z | HiFi 4 | HiFi 5 |                         |
| 16          | 1   | 48                | 47.7                   | 48.6   | 42.9    | 41.3   | 37.5   | SILK VBR                |
| 16          | 1   | 48                | 66.1                   | 67.6   | 59.5    | 57.5   | 52.5   | SILK VBR FEC 10% loss   |
| 24          | 1   | 40                | 56.2                   | 57.6   | 50.9    | 48.5   | 43.7   | Hybrid VBR              |
| 24          | 1   | 40                | 73.1                   | 75.1   | 66.1    | 63.4   | 57.6   | Hybrid VBR FEC 10% loss |
| 48          | 2   | 128               | 13.6                   | 15.2   | 13.0    | 12.4   | 10.6   | CELT VBR Complexity 0   |
| 48          | 2   | 128               | 25.3                   | 28.1   | 24.2    | 23.3   | 19.5   | CELT VBR Complexity 7   |
| 48          | 2   | 128               | 44.8                   | 50.8   | 43.4    | 42.0   | 35.9   | CELT VBR Complexity 10  |
| 48          | 8   | 256               | 123.9                  | 135.6  | 118.1   | 112.1  | 94.4   | CELT VBR Complexity 10  |

### Decoder

Table 1-5 Decoder Timings

| Rate<br>kHz | Nch | Bitrate<br>(kbps) | Average CPU Load (MHz) |        |         |        |        |                                                         |
|-------------|-----|-------------------|------------------------|--------|---------|--------|--------|---------------------------------------------------------|
|             |     |                   | HiFi 1                 | HiFi 3 | HiFi 3z | HiFi 4 | HiFi 5 |                                                         |
| 16          | 1   | 48                | 5.4                    | 6.4    | 5.4     | 5.2    | 4.3    | SILK (20 ms)                                            |
| 24          | 1   | 40                | 6.1                    | 6.4    | 5.7     | 5.3    | 4.7    | Hybrid (20 ms)                                          |
| 48          | 2   | 128               | 13.2                   | 15.6   | 13.3    | 13.0   | 10.6   | CELT (20 ms)                                            |
| 48          | 2   | 183.6             | 26.4                   | 31.2   | 25.4    | 25.4   | 21.6   | CELT (variable 2.5-20ms)                                |
| 48          | 6   | 392               | 42.9                   | 50.8   | 43.2    | 52.2   | 35.1   | Multistream (20 ms, 4 streams - 2 coupled, 2 uncoupled) |

## Ogg Parser

Table 1-6 Ogg Parser Timings

| Rate<br>kHz | Nch | Bitrate<br>(kbps) | Average CPU Load (MHz) |        |         |        |        |                          |
|-------------|-----|-------------------|------------------------|--------|---------|--------|--------|--------------------------|
|             |     |                   | HiFi 1                 | HiFi 3 | HiFi 3z | HiFi 4 | HiFi 5 |                          |
| 48          | 2   | 183.6             | 0.4                    | 0.5    | 0.4     | 0.4    | 0.4    | CELT (variable 2.5-20ms) |

## Ogg Packer

Table 1-7 Ogg Packer Timings

| Rate<br>kHz | Nch | Bitrate<br>(kbps) | Average CPU Load (MHz) |        |         |        |        |             |
|-------------|-----|-------------------|------------------------|--------|---------|--------|--------|-------------|
|             |     |                   | HiFi 1                 | HiFi 3 | HiFi 3z | HiFi 4 | HiFi 5 |             |
| 48          | 8   | 256               | 0.4                    | 0.6    | 0.5     | 0.5    | 0.4    | CELT (20ms) |

**Note** Performance specification measurements are carried out on a cycle-accurate simulator assuming an ideal memory system, that is, one with zero memory wait states. This is equivalent to running with all code and data in local memories or using an infinite-size, pre-warmed cache model.

**Note** The performance numbers of the Opus encoder and decoder libraries are similar to those of the Opus-SILK and Opus-CELT encoder and decoder libraries for the corresponding test streams.

**Note** The MCPS numbers for HiFi 3z/HiFi 4 are obtained by running the test that is recompiled from the HiFi 3 optimized source code in the HiFi 3z/HiFi 4 configuration. No specific optimization is done for HiFi 3z/HiFi 4. Additional optimization is done for HiFi 1 and HiFi5.

**Note** All the memory and MCPS numbers are captured with the XT-CLANG compiler and the RJ-2024.3 tools.

**Note** Encoder performance was measured with the corresponding encoding mode set using the “–force\_mode” command line option.

**Note** Encoder performance for SILK and Hybrid vectors was measured for the highest complexity configuration parameter (complexity = 10). Encoder performance for all vectors was measured for a frame size of 20 ms.

## 2. Generic HiFi Speech Codec API

This section describes the APIs common to all the HiFi speech codec libraries. The APIs facilitate any codec that works in the overall method shown in the Figure 1.

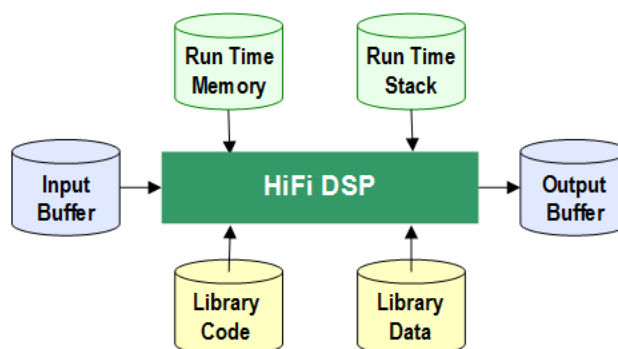


Figure 1 HiFi Speech Codec Interfaces

Section 2.1 discusses the types of runtime memories required by the codecs. This includes input-output buffers, persistent memory, stack, and scratch memories. The APIs are designed in such a way that the codec is reentrant. The APIs are implemented in a way that the application does not need to consider the codec implementation details.

### 2.1 Memory Management

The HiFi speech codec APIs support a flexible memory scheme and a simple interface that simplifies integration into the final application. The APIs allow the codecs to request the required memory for their operations during runtime.

The run-time memory requirement mainly consists of scratch and persistent memory. The codecs also require an input buffer and output buffer for data transfers into and out of the codec.

#### 2.1.1 API Handle/Persistent Memory

The codec state information is stored in persistent memory in a structure referenced via an opaque handle. The application passes the handle for each API call. This object contains all state and history information maintained from one codec frame invocation to the next within the same thread or instance. The codec expects that the contents of the persistent memory remain unchanged by the system, apart from the codec library itself, for the complete lifetime of the codec operation.

## 2.1.2 Scratch Memory

This is the temporary buffer used by the codec for processing. The contents of this memory region should be unchanged if the actual codec process is running, that is, if the thread running the codec is inside any API call. The system can use this region freely between successive calls to the codec.

## 2.1.3 Input Buffer

This is the buffer used by the algorithm to accept input data. Before the call to the codec, the input buffer must be completely filled with input data.

## 2.1.4 Output Buffer

This is the buffer in which the algorithm writes the output. This buffer needs to be made available for the codec before the frame processing begins. The application can change the output buffer pointer between calls to the codec. This allows the codec to write directly to the required output area. Sufficient size should be allocated for the output buffer as requested by the codec library.

# 2.2 C Language APIs

The speech codec APIs consist of query, initialization, and process functions. In the naming scheme below, `<codec>` is either the codec name or the codec name with an `_enc` or `_dec` suffix for the encoder and decoder-specific functions, respectively.

### Query Functions: `xa_<codec>_get_<data>`

Query functions are used in the startup and memory allocation codec stages to obtain information about the version and memory requirements of the codec library.

In the naming scheme, `<data>` is used to specify the type of information requested from the codec library.

### Initialization Functions: `xa_<codec>_init`

Initialization functions are used to reset the codec to its initial state. Because the codec library is fully reentrant, a process can initialize the codec library multiple times, and multiple processes can initialize the same codec library as required.

### Process Functions: `xa_<codec>`

Process functions are used to encode and/or decode speech frames.

The codec sequence and the functions associated with each stage are described in detail in Section 3.

Figure 2 shows an overview of the codec flow.

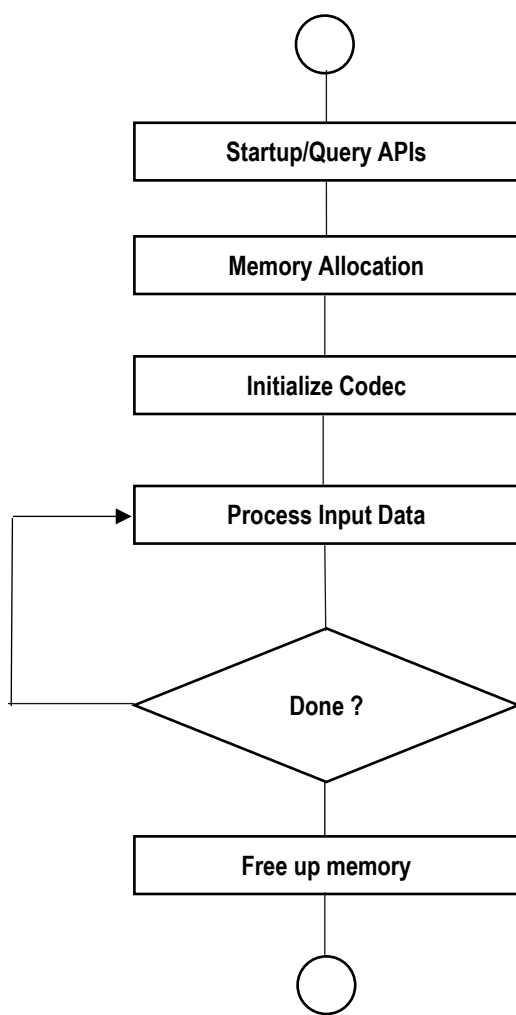


Figure 2 Speech Codec Flow Overview

## 2.3 About Generic API Errors

Speech codec API functions return the `XA_ERRORCODE` that is of `signed int` type. Table 2-1 defines the format of the error codes.

Table 2-1 Error Codes Format

| 31    | 30-15    | 14 – 11 | 10 – 6 | 5 – 0    |
|-------|----------|---------|--------|----------|
| Fatal | Reserved | Class   | Codec  | Sub code |

The errors that are returned from the APIs are divided into fatal and nonfatal errors based on severity. Fatal errors require restarting the codec, while nonfatal errors provide information to the application.

Errors can be further classified into the following three classes:

- API errors occur when the API is used incorrectly.
- Config errors occur when the codec parameters are incorrect or not supported.
- Process Errors occur during the primary encoding or decoding and indicate issues arising due to invalid or bad input data.

The subcode specifies the exact condition which triggers the error.

The following are fatal generic error codes that should not occur during normal application operations. They indicate a serious error in the application that is calling the codec.

- `XA_API_FATAL_MEM_ALLOC`  
At least one of the pointers passed into the API function is NULL.
- `XA_API_FATAL_MEM_ALIGN`  
At least one of the pointers passed into the API function is not properly aligned.

## ***2.4 Files Supporting the APIs***

The following are the common include files (`include`):

- `xa_error_standards.h`  
Includes macros and definitions for all the generic errors.
- `xa_type_def.h`  
Includes the definitions of all datatypes required for the API calls.
- `xa_error_handler.h`  
Includes definitions used by error handling functions.

## 3. HiFi DSP Opus Codec API

The HiFi DSP Opus Codec conforms to the generic speech codec API and the codec flow shown in Figure 2. The following sections describe the Opus codec-specific files, the supported I/O formats, DTX modes, bit rates, and API functions.

### 3.1 Opus Codec-specific Files

The Opus codec parameter header file (`include/opus_codec`):

- `xa_opus_codec_api.h`

The Opus codec library (`lib`):

- `xa_opus_codec.a`
- `xa_opus_dec.a`
- `xa_opus_enc.a`

### 3.2 I/O Formats

The encoder's input is a block of 16-bit speech samples representing 2.5 ... 120 ms of speech frame data. The Opus encoder can have a maximum of six frames per packet. The encoder can accept input signals of the following sampling frequencies: 8 kHz, 12 kHz, 16 kHz, 24 kHz, and 48 kHz.

The decoder processes the encoded bitstream frame-by-frame. Under normal conditions, one complete packet with an additional 4 bytes at the beginning is required in the input buffer. These 4 bytes should contain the 'range decoder state' variable before each packet in the bitstream is encoded from the conformant Opus encoder. In cases where these additional 4 bytes are not available or if the application wants to skip this check, it can be done through the 'no\_range\_dec\_state' decoder control structure element described in Table 3-2. In case of a lost packet, the decoder extracts the FEC data from the next packet and processes it. Thus, in case of packet loss, the application puts two packets in the input buffer for FEC to operate correctly. The decoder outputs 2.5 ... 120 ms frames composed of 16-bit PCM samples. The decoder can generate output signals of the following sampling frequencies: 8 kHz, 12 kHz, 16 kHz, 24 kHz, and 48 kHz.

### 3.3 Control Structure for Opus Codec

The encoder control structure `xa_opus_enc_control_t` is used to communicate various parameters listed in Table 3-1 between the codec library and the application.

**Note** All the parameters are of the **WORD32** data type.

Table 3-1 Encoder Control Structure `xa_opus_enc_control_t` Parameters

| Parameter                    | Input/<br>Output | Default<br>value | Description                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------|------------------|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>application</code>     | input            |                  | Application coding mode.<br>Supported configurations: <ul style="list-style-type: none"> <li>- <code>XA_OPUS_APPLICATION_VOIP</code></li> <li>- <code>XA_OPUS_APPLICATION_AUDIO</code></li> <li>- <code>XA_OPUS_APPLICATION_RESTRICTED_LOWDELAY</code></li> </ul> <b>Note:</b> This parameter is not configurable in the OPUS-SILK library.                                                                                           |
| <code>API_sampleRate</code>  | input            |                  | Input signal sampling rate in Hz.<br>Described in Section 1.3.                                                                                                                                                                                                                                                                                                                                                                        |
| <code>API_numChannels</code> | input            |                  | Number of channels in input signal; 1 to 8.                                                                                                                                                                                                                                                                                                                                                                                           |
| <code>bitRate</code>         | input            |                  | Bitrate during active speech in bps.<br>The recommended range is 6000 to 510000, as described in Section 1.3. All positive values and -1 (maximum possible bitrate based on other parameters) are accepted.                                                                                                                                                                                                                           |
| <code>cbr</code>             | input            | 0                | Flag to enable constant bitrate, 0/1.                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>cvbr</code>            | input            | 0                | Flag to enable constrained variable bitrate, 0/1.                                                                                                                                                                                                                                                                                                                                                                                     |
| <code>bandwidth</code>       | input            | 0                | Audio bandwidth (from narrowband to fullband).<br>Supported configurations: <ul style="list-style-type: none"> <li>- <code>XA_OPUS_AUTO (0)</code></li> <li>- <code>XA_OPUS_BANDWIDTH_NARROWBAND</code></li> <li>- <code>XA_OPUS_BANDWIDTH_MEDIUMBAND</code></li> <li>- <code>XA_OPUS_BANDWIDTH_WIDEBAND</code></li> <li>- <code>XA_OPUS_BANDWIDTH_SUPERWIDEBAND</code></li> <li>- <code>XA_OPUS_BANDWIDTH_FULLBAND</code></li> </ul> |
| <code>max_bandwidth</code>   | input            |                  | Audio bandwidth (from narrowband to fullband).<br>Supported configurations: <ul style="list-style-type: none"> <li>- <code>XA_OPUS_AUTO (0)</code></li> <li>- <code>XA_OPUS_BANDWIDTH_NARROWBAND</code></li> <li>- <code>XA_OPUS_BANDWIDTH_MEDIUMBAND</code></li> <li>- <code>XA_OPUS_BANDWIDTH_WIDEBAND</code></li> <li>- <code>XA_OPUS_BANDWIDTH_SUPERWIDEBAND</code></li> <li>- <code>XA_OPUS_BANDWIDTH_FULLBAND</code></li> </ul> |

| Parameter                                                                           | Input/<br>Output | Default<br>value | Description                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------|------------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| max_payload                                                                         | input            | 1500             | Max payload size in bytes; [1, XA_OPUS_MAX_BYTES_PER_PACKET-4] range (values outside range ignored).                                                                                                                                            |
| complexity                                                                          | input            | 10               | Complexity mode (0-10): 0 is lowest, 5 is medium, and 10 is highest complexity.                                                                                                                                                                 |
| SILK_InBandFEC_enabled                                                              | input            | 0                | Flag to enable SILK in-band Forward Error Correction (0/1). By default, it is disabled.<br><b>Note:</b> This parameter is not configurable in the OPUS-CELT library.                                                                            |
| force_numChannels                                                                   | input            | 0                | Force number of output channels (0/1/2). Default is forcing disabled (0).                                                                                                                                                                       |
| SILK_DTX_enabled                                                                    | input            | 0                | Flag to enable discontinuous transmission (DTX); 0/1 Default DTX is disabled.<br>0–disable, 1–enable.<br><b>Note:</b> This parameter is not configurable in the OPUS-CELT library.                                                              |
| packetLossPercentage                                                                | input            | 0                | Uplink packet loss in percent (0-100).                                                                                                                                                                                                          |
| force_mode                                                                          | input            | 0                | Force encode mode.<br>Supported configurations:<br>- XA_OPUS_AUTO (0)<br>- XA_OPUS_MODE_SILK_ONLY<br>- XA_OPUS_MODE_HYBRID<br>- XA_OPUS_MODE_CELT_ONLY<br><b>Note:</b> This parameter is not configurable in OPUS-SILK and OPUS-CELT libraries. |
| signal_type                                                                         | input            | 0                | Configures the type of signal being encoded.<br>Supported configurations:<br>- XA_OPUS_AUTO (0)<br>- XA_OPUS_SIGNAL_VOICE<br>- XA_OPUS_SIGNAL_MUSIC<br><b>Note:</b> This parameter is not configurable in OPUS-SILK and OPUS-CELT libraries.    |
| reset_state                                                                         | input/<br>output | 0                | Resets the encoder state to be equivalent to a freshly initialized state, 0/1.<br>0 – no reset<br>1 – reset the encoder<br><b>Note:</b> Non-zero values are treated as 1.                                                                       |
| variable_duration                                                                   | Input            | 0                | Deprecated. Not used by the library.                                                                                                                                                                                                            |
| The following elements are required for Ogg container/multi-stream encoder support. |                  |                  |                                                                                                                                                                                                                                                 |
| nb_streams                                                                          | Input/<br>Output | 0                | Total number of streams, 1 – XA_OPUS_MAX_NUM_STREAMS, should be set to 1 for raw opus streams.                                                                                                                                                  |

| Parameter                                         | Input/<br>Output | Default<br>value | Description                                                                                                                                                                                                                                                                                                                     |
|---------------------------------------------------|------------------|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>nb_coupled</code>                           | Input/<br>output | 0                | Total number of coupled streams, 0 – nb_streams-1.<br>For raw opus streams: it should be set to 0 for mono decoding and 1 for stereo decoding.                                                                                                                                                                                  |
| <code>channel_mapping</code>                      | Input            | 0                | Channel mapping family.<br>0: RTP mapping <sup>2</sup> , allowed number of channels 1 or 2.<br>1: Vorbis channel mapping <sup>3</sup> , allowed number of channels 1 to 8.<br>255: No defined channel meaning, allowed number of channels 1 to 8.<br>(Ref: RFC7845 - Section 5.1.1)<br>Should be set to 0 for raw Opus streams. |
| <code>stream_map[XA_OPUS_MAX_NUM_CHANNELS]</code> | Input/<br>output | 0                | Output channel map (ignored if channel_mapping is 0);<br>Array of XA_OPUS_MAX_NUM_CHANNELS unsigned chars with range -<br>0 to (nb_streams+nb_coupled-1): route corresponding stream (for interpretation of channels, refer to RFC7845-Section 5.1.1.2).<br>255: output silence (Ref: RFC7845 - Section 5.1.1).                 |
| <code>skip</code>                                 | input/<br>output | 0                | Skip bytes to be used for delay compensation.                                                                                                                                                                                                                                                                                   |
| <code>non_std</code>                              | Input            | 0                | Flag, when enabled, allows certain behavior that is not compliant with the standard. For example, processing of the last stream as a non-LFE stream, as mentioned below.<br>Mapping family 1 assumes the last channel to be LFE for 6 and more channel input. To disable this condition, the non_std flag should be set to 1.   |

The decoder control structure `xa_opus_dec_control_t` communicates various parameters listed in Table 3-2 between the codec library and the application.

---

**Note** All the parameters are of the **WORD32** data type except for `stream_map` that is an array of **UWORD8**.

---

<sup>2</sup> RFC7845 Section 5.1.1.1

<sup>3</sup> RFC7845 Section 5.1.1.2

Table 3-2 Decoder Control Structure xa\_opus\_dec\_control\_t Parameters

| Parameter                                                                                                                       | Input/<br>Output | Default<br>value | Description                                                                                                                                                                                                                                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------|------------------|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API_sampleRate                                                                                                                  | input            |                  | Output signal sampling rate in Hz.<br>Described in Section 1.3.                                                                                                                                                                                                                                                        |
| API_numChannels                                                                                                                 | input            |                  | Number of channels of output signal; 1/2 if<br>channel_mapping is 0, 1-<br>XA_OPUS_MAX_NUM_CHANNELS if<br>channel_mapping is 1.                                                                                                                                                                                        |
| SILK_InBandFEC_enabled                                                                                                          | input            | 0                | Flag to enable SILK in-band Forward Error<br>Correction (0/1).<br>0—disable. 1—enable.<br><b>Note:</b> This parameter is not configurable in the<br>OPUS-CELT library.                                                                                                                                                 |
| gain                                                                                                                            | input            | 0                | Decoder gain adjustment, [-32768, 32767] range.<br>$gain = \text{pow}(10, x/(20.0*256))$                                                                                                                                                                                                                               |
| lostFlag                                                                                                                        | input            | 0                | Packet loss indicator,<br>0 - no loss, 1 – loss                                                                                                                                                                                                                                                                        |
| framesPerPacket                                                                                                                 | output           |                  | Frames per packet 1, 2, 3, 4, 5, 6.                                                                                                                                                                                                                                                                                    |
| moreInternalDecoder<br>Frames                                                                                                   | output           |                  | Flag to indicate that the decoder has remaining<br>payloads internally.<br>This flag will be set to one if the packet still has<br>some bytes to be decoded. It will be set to zero<br>after decoding all the frames in the packet.<br>The application shall load the next packet only when<br>this flag becomes zero. |
| reset_state                                                                                                                     | input/<br>output | 0                | This parameter is currently not used by the<br>decoder and will not affect decoding.                                                                                                                                                                                                                                   |
| no_range_dec_state                                                                                                              | input            | 0                | 'Range decoder state' conformance check variable<br>control (Ref: RFC6716 - Section 6)<br>1: Excluded 'Range decoder state' variable in input<br>payload.<br>0: Included 'Range decoder state' variable in input<br>payload.                                                                                           |
| The following elements are required for Ogg container/multistream decode support and can be parsed<br>from a valid Opus header. |                  |                  |                                                                                                                                                                                                                                                                                                                        |
| version                                                                                                                         | input            | 0                | Bitstream version from Opus Header. Only major<br>version 0 streams are supported.                                                                                                                                                                                                                                     |
| nb_streams                                                                                                                      | Input            | 0                | Total number of streams, 1 –<br>XA_OPUS_MAX_NUM_STREAMS, should be set<br>to 1 for raw opus streams.                                                                                                                                                                                                                   |

| Parameter                                         | Input/<br>Output | Default<br>value | Description                                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------------------|------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>nb_coupled</code>                           | Input            | 0                | Total number of coupled streams, 0 – <code>nb_streams-1</code> ,<br>For raw opus streams, this value should be set to 0 for mono decoding, and 1 for stereo decoding.                                                                                                                                                                                  |
| <code>channel_mapping</code>                      | Input            | 0                | Channel mapping family;<br>0: RTP mapping <sup>4</sup> , allowed number of channels 1 or 2.<br>1: Vorbis channel mapping <sup>5</sup> , allowed number of channels 1 to 8.<br>255: No defined channel meaning, allowed number of channels 1 to 8.<br>(Ref: RFC7845 - Section 5.1.1)                                                                    |
| <code>stream_map[XA_OPUS_MAX_NUM_CHANNELS]</code> | Input            | 0                | Output channel map (ignored if <code>channel_mapping</code> is 0); Array of <code>XA_OPUS_MAX_NUM_CHANNELS</code> unsigned chars with range -<br>0 to ( <code>nb_streams+nb_coupled-1</code> ): route corresponding stream (for interpretation of channels, refer to RFC7845- Section 5.1.1.2),<br>255: output silence (Ref: RFC7845 - Section 5.1.1). |

## 3.4 API Functions

The following sections specify the Opus codec-specific API functions relevant to each stage in the codec flow.

### 3.4.1 Startup Stage

The API startup functions described in Table 3-3, get the various identification strings from the codec library. They are for information only, and their usage is optional. These functions do not take any input arguments and return `const char *`.

Table 3-3 Library Identification Functions

| Function                                        | Description               |
|-------------------------------------------------|---------------------------|
| <code>xa_opus_get_lib_name_string</code>        | Gets the library name.    |
| <code>xa_opus_get_lib_version_string</code>     | Gets the library version. |
| <code>xa_opus_get_lib_api_version_string</code> | Gets the API version.     |

<sup>4</sup> RFC7845 Section 5.1.1.1

<sup>5</sup> RFC7845 Section 5.1.1.2

**Example**

```

const char *name = xa_opus_get_lib_name_string();
const char *ver = xa_opus_get_lib_version_string();
const char *apiver = xa_opus_get_lib_api_version_string();

```

**Errors**

- None

## 3.4.2 Memory Allocation Stage

During the memory allocation stage, the application must reserve the necessary memory for the Opus Encoder and Decoder API handles (persistent state) and scratch buffers. The required alignment of the handles and the scratch buffers is 8 bytes. The application can use the functions listed in Table 3-4 to query the codec library for the required size of each buffer.

The codec requires input and output frame buffers for its operation. However, these buffers do not need to be reserved at this stage. Instead, pointers to the frame buffers are passed in each invocation of the codec's process function. The size and alignment requirements of the I/O buffers are specified in Section 3.4.4.

Table 3-4 Memory Management Functions

| Function                                          | Description                                                                                                                                                                                      |
|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>xa_opus_enc_get_handle_byte_size</code>     | Returns the size of the Opus Encoder API handle (persistent state) in bytes.                                                                                                                     |
| <code>xa_opus_dec_get_handle_byte_size</code>     | Returns the size of the Opus Decoder API handle (persistent state) in bytes.                                                                                                                     |
| <code>xa_opus_enc_get_scratch_byte_size</code>    | Returns the size of the Opus Encoder scratch buffer in bytes.                                                                                                                                    |
| <code>xa_opus_dec_get_scratch_byte_size</code>    | Returns the size of the Opus Decoder scratch buffer in bytes.                                                                                                                                    |
| <code>xa_opus_mch_enc_get_handle_byte_size</code> | Returns the size of the Opus Encoder API handle (persistent state) in bytes.<br>Replacement function for <code>xa_opus_enc_get_handle_byte_size()</code> if Mapping Family 255 needs to be used. |

**Example**

```

WORD32 enc_handle_size, dec_handle_size;
WORD32 enc_scratch_size, dec_scratch_size;
enc_handle_size = xa_opus_enc_get_handle_byte_size(
    WORD32 numChannels);
dec_handle_size = xa_opus_dec_get_handle_byte_size(
    WORD32 nb_streams, WORD32 nb_coupled);
enc_handle_size      = xa_opus_mch_enc_get_handle_byte_size  (WORD32
numChannels, WORD32 channel_mapping);

```

For the Opus Celt library,

```
enc_scratch_size = xa_opus_enc_get_scratch_byte_size (
    WORD32 numChannels,
    WORD32 sampleRate,
    WORD32 application);
```

For Opus Silk and Opus libraries,

```
enc_scratch_size = xa_opus_enc_get_scratch_byte_size (
    WORD32 numChannels,
    WORD32 sampleRate);

dec_scratch_size = xa_opus_dec_get_scratch_byte_size(
    WORD32 channel_mapping);
```

## Errors

- None

## 3.4.3 Initialization Stage

In the initialization stage, the application points the Opus codec to its API handle. The application also specifies various other parameters related to the codec's operation and places the codec in its initial state. The API functions for Opus encoder and decoder initialization are specified in Table 3-5 and Table 3-6, respectively.

### Opus Encoder's Initialization Function

Table 3-5 Opus Encoder's Initialization Function

|                    |                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>    | <code>xa_opus_enc_init</code>                                                                                                                                                                                                                                                                                                                                                  |
| <b>Syntax</b>      | <pre>XA_ERRORCODE xa_opus_enc_init (xa_codec_handle_t handle,     xa_opus_enc_control_t *enc_control);</pre>                                                                                                                                                                                                                                                                   |
| <b>Description</b> | Resets the encoder API handle to its initial state. Sets up the encoder using the supplied configuration. It also returns the current configuration of the encoder in the <code>enc_control</code> structure. You can modify this structure later before the processing call to control the encoder's behavior.                                                                |
| <b>Parameters</b>  | <p><b>Input:</b> <code>handle</code><br/> Pointer to the encoder handle (persistent state).<br/> Required size: see <code>xa_opus_enc_get_handle_byte_size</code><br/> Required alignment: 8 bytes</p> <p><b>Input:</b> <code>enc_control</code><br/> Pointer to the encoder control structure.<br/> Refer to <code>xa_opus_enc_control_t</code> described in Section 3.3.</p> |

## Example

```
handle = (pVOID) malloc(handle_size);
xa_opus_enc_control_t enc_control;

error_code = xa_opus_enc_init (handle,
                              &enc_control);
```

## Errors

- XA\_API\_FATAL\_MEM\_ALLOC
- XA\_API\_FATAL\_MEM\_ALIGN
- XA\_OPUS\_CONFIG\_FATAL\_SAMP\_FREQ\_NOT\_SUPPORTED  
The sampling frequency is not supported.
- XA\_OPUS\_CONFIG\_FATAL\_NUM\_CHANNEL\_NOT\_SUPPORTED  
The number of channels is not supported.
- XA\_OPUS\_CONFIG\_FATAL\_ENC\_INVALID\_APP\_SETTING  
Application mode is not supported.
- XA\_OPUS\_CONFIG\_FATAL\_ENC\_INVALID\_BITRATE\_SETTING  
Invalid bit rate.
- XA\_OPUS\_CONFIG\_FATAL\_INVALID\_PARAM\_COMBINATION  
Invalid configuration parameter combination. (Not used)
- XA\_OPUS\_EXECUTE\_FATAL\_INIT\_FAILURE  
An error occurred in initializing the encoder.
- XA\_OPUS\_CONFIG\_NONFATAL\_ENC\_INVALID\_FORCE\_CH\_SETTING  
Invalid encoder channel force setting.
- XA\_OPUS\_CONFIG\_NONFATAL\_ENC\_INVALID\_MAX\_BANDWIDTH\_SETTING  
Invalid encoder maximum bandwidth setting.
- XA\_OPUS\_CONFIG\_NONFATAL\_ENC\_INVALID\_BANDWIDTH\_SETTING  
Invalid encoder bandwidth setting.
- XA\_OPUS\_CONFIG\_NONFATAL\_ENC\_INVALID\_COMPLEXITY\_SETTING  
Invalid encoder complexity setting.
- XA\_OPUS\_CONFIG\_NONFATAL\_ENC\_INVALID\_PACKET\_LOSS\_PERC\_SETTING  
Invalid encoder expected packet loss percentage setting.

- `XA_OPUS_CONFIG_NONFATAL_ENC_INVALID_FORCE_MODE_SETTING`  
Invalid encoder force mode setting.
- `XA_OPUS_CONFIG_NONFATAL_ENC_INVALID_SIGNAL_TYPE_SETTING`  
Invalid encoder signal type setting.
- `XA_OPUS_EXECUTE_NONFATAL_BAD_ARG`  
Invalid argument setting.
- `XA_OPUS_CONFIG_FATAL_ENC_NUM_STREAMS_NOT_SUPPORTED`  
The number of streams is not supported by the encoder.
- `XA_OPUS_CONFIG_FATAL_ENC_UNKNOWN_CHANNEL_MAPPING`  
The channel mapping family is not supported by the encoder.

## Opus Decoder's Initialization Function

Table 3-6 Opus Decoder's Initialization Function

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>    | <code>xa_opus_dec_init</code>                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Syntax</b>      | <code>XA_ERRORCODE</code><br><code>xa_opus_dec_init (</code><br><code>xa_codec_handle_t handle,</code><br><code>xa_opus_dec_control_t *dec_control)</code>                                                                                                                                                                                                                                                                       |
| <b>Description</b> | Resets the decoder API handle to its initial state. Sets the decoder to run using the supplied configuration in the specified encoded speech format.                                                                                                                                                                                                                                                                             |
| <b>Parameters</b>  | <p><b>Input:</b> <code>handle</code><br/>           Pointer to the decoder handle (persistent state).<br/>           Required size: see <code>xa_opus_dec_get_handle_byte_size</code><br/>           Required alignment: 8 bytes</p> <p><b>Input:</b> <code>dec_control</code><br/>           Pointer to the decoder control structure.<br/>           Refer to <code>xa_opus_dec_control_t</code> described in Section 3.3.</p> |

### Example

```
dec_handle = (pVOID)malloc(handle_size);
xa_opus_dec_control_t dec_control;
error_code = xa_opus_dec_init(dec_handle,
                             &dec_control);
```

## Errors

- **XA\_API\_FATAL\_MEM\_ALLOC**
- **XA\_API\_FATAL\_MEM\_ALIGN**
- **XA\_OPUS\_EXECUTE\_FATAL\_INIT\_FAILURE**  
Opus decoder init failure.
- **XA\_OPUS\_CONFIG\_FATAL\_SAMP\_FREQ\_NOT\_SUPPORTED**  
Not valid setting of sampling frequency.
- **XA\_OPUS\_CONFIG\_FATAL\_NUM\_CHANNEL\_NOT\_SUPPORTED**  
The number of channels is not supported.
- **XA\_OPUS\_CONFIG\_FATAL\_INVALID\_PARAM\_COMBINATION**  
Invalid configuration parameter combination. Indicates that either `channel_mapping` and `API_numChannels` values are incompatible or `nb_coupled` is more than `nb_streams`.
- **XA\_OPUS\_CONFIG\_FATAL\_DEC\_NUM\_STREAMS\_NOT\_SUPPORTED**  
The number of streams is not supported by the decoder.
- **XA\_OPUS\_CONFIG\_FATAL\_DEC\_OPUS\_BITSTREAM\_VERSION\_NOT\_SUPPORTED**  
Bitstream version not supported (Only major version 0 is supported).
- **XA\_OPUS\_CONFIG\_FATAL\_DEC\_UNKNOWN\_CHANNEL\_MAPPING**  
Unknown channel mapping family.
- **XA\_OPUS\_CONFIG\_FATAL\_DEC\_BAD\_STREAM\_MAP**  
The stream map is not valid for the given input stream.
- **XA\_OPUS\_CONFIG\_NONFATAL\_DEC\_INVALID\_GAIN\_SETTING**  
Invalid decoder gain adjustment setting.
- **XA\_OPUS\_CONFIG\_NONFATAL\_DEC\_STREAM\_MAP\_IGNORED**  
Output stream map ignored.

### 3.4.4 Process Stage

The Opus codec processes the input stream and generates the output stream frame-by-frame. Each call to a codec's processing function requires one complete frame as input and produces one complete packet as output.

If the input buffer contains insufficient data, the Opus codec does not consume any bytes and returns a non-fatal error that allows the application to fill in additional data before calling the decoder again.

The Opus encoder takes as input one “2.5 ... 120” ms speech frame containing 16-bit samples. The Opus decoder also processes data of a “2.5 ... 120” ms frame. It has a buffer mechanism for extracting FEC data from the next packet. It holds data for the next two packets in the input buffer.

Table 3-7 lists the macros used to calculate I/O buffer sizes and configuration of Opus.

Table 3-7 Macros to Calculate I/O Buffer Sizes and Configuration

| Defines with Values                                                                                                                                                       | Description                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>XA_OPUS_MAX_NUM_STREAMS 8</code>                                                                                                                                    | Maximum number of streams supported by the decoder and encoder.                                                 |
| <code>XA_OPUS_MAX_NUM_CHANNELS 8</code>                                                                                                                                   | Maximum number of channels supported by the decoder and encoder.                                                |
| <code>XA_OPUS_MAX_FRAMES_PER_PACKET 6</code>                                                                                                                              | Maximum 6 20 ms frames (total 120 ms) per packet.                                                               |
| <code>XA_OPUS_MAX_SAMPLES_PER_FRAME_CELT 960</code>                                                                                                                       | Maximum number of samples in a frame in the CELT mode.                                                          |
| <code>XA_OPUS_MAX_BYTES_CHANNEL_PACKET</code><br><br><code>XA_OPUS_MAX_SAMPLES_PER_FRAME_CELT * sizeof(WORD16) *</code><br><br><code>XA_OPUS_MAX_FRAMES_PER_PACKET</code> | Maximum number of bytes in one channel of a packet.                                                             |
| <code>XA_OPUS_MAX_FEC_DELAY 1</code>                                                                                                                                      | Latency in packets in case of data loss.                                                                        |
| <code>XA_OPUS_MAX_PACKET_IN_INP_BUF</code><br><br><code>XA_OPUS_MAX_FEC_DELAY + 1</code>                                                                                  | Number of packets in input buffer in case of packet loss.                                                       |
| <code>XA_OPUS_MAX_BYTES_ENCODED_PACKET 1504</code>                                                                                                                        | Maximum number of bytes in one encoded packet / maximum number of bytes in input buffer if no packets are lost. |
| <code>XA_OPUS_MAX_DEC_INP_BYTES</code><br><br><code>XA_OPUS_MAX_BYTES_ENCODED_PACKET *</code><br><br><code>XA_OPUS_MAX_PACKET_IN_INP_BUF</code>                           | Max. number of data bytes in input buffer in case of packet loss.                                               |

| Defines with Values                          | Description                                                                                                          |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| XA_OPUS_APPLICATION_VOIP 2048                | Setting for most VoIP/videoconference applications, where the listening quality and intelligibility matter the most. |
| XA_OPUS_APPLICATION_AUDIO 2049               | Setting for broadcast/high-fidelity application where the decoded audio should be as close as possible to the input. |
| XA_OPUS_APPLICATION_RESTRICTED_LOWDELAY 2051 | Setting is only used when the lowest-achievable latency is what matters most. Voice-optimized modes cannot be used.  |
| XA_OPUS_AUTO 0                               | Value of auto/default setting                                                                                        |
| XA_OPUS_BANDWIDTH_NARROWBAND 1101            | 4 kHz bandwidth                                                                                                      |
| XA_OPUS_BANDWIDTH_MEDIUMBAND 1102            | 6 kHz bandwidth                                                                                                      |
| XA_OPUS_BANDWIDTH_WIDEBAND 1103              | 8 kHz bandwidth                                                                                                      |
| XA_OPUS_BANDWIDTH_SUPERWIDEBAND 1104         | 12 kHz bandwidth                                                                                                     |
| XA_OPUS_BANDWIDTH_FULLBAND 1105              | 20 kHz bandwidth                                                                                                     |
| XA_OPUS_MODE_SILK_ONLY 1000                  | SILK mode                                                                                                            |
| XA_OPUS_MODE_HYBRID 1001                     | Hybrid mode                                                                                                          |
| XA_OPUS_MODE_CELT_ONLY 1002                  | CELT mode                                                                                                            |
| XA_OPUS_SIGNAL_VOICE 3001                    | The signal being encoded is voice.                                                                                   |
| XA_OPUS_SIGNAL_MUSIC 3002                    | The signal being encoded is music.                                                                                   |

The syntax of the Opus encoder's and decoder's process functions is specified in Table 3-8 and Table 3-9, respectively.

## Opus Encoder's Process Function

Table 3-8 Opus Encoder's Process Function

|                    |                                                                                                                                                                                                                                  |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>    | xa_opus_enc                                                                                                                                                                                                                      |
| <b>Syntax</b>      | <pre> XA_ERRORCODE xa_opus_enc (     xa_codec_handle_t handle,     pWORD16 inp_speech,     pUWORD8 enc_speech,     WORD16 inp_samples,     xa_opus_enc_control_t *enc_control,     WORD16 *out_bytes,     pVOID scratch); </pre> |
| <b>Description</b> | Encodes one frame of data.                                                                                                                                                                                                       |
| <b>Parameters</b>  | <b>Input:</b> handle<br>Pointer to the encoder handle (persistent state).<br>8 bytes alignment is required.                                                                                                                      |

|             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|             | <p><b>Input:</b> <code>inp_speech</code><br/> Pointer to the input speech PCM samples. The input must contain at least one frame of data.<br/> 8 bytes alignment is required.</p> <p><b>Output:</b> <code>enc_speech</code><br/> Pointer to the encoded speech parameters (Opus core frame).<br/> 1-byte alignment is required.</p> <p><b>Input:</b> <code>inp_samples</code><br/> Number of samples in the input speech buffer.</p> <p><b>Input/Output:</b> <code>enc_control</code><br/> Pointer to the encoder control structure.<br/> Refer to <code>xa_opus_enc_control_t</code> described in Section 3.3.</p> <p><b>Output:</b> <code>out_bytes</code><br/> Number of output bytes generated by the encoder.</p> <p><b>Input/Output:</b> <code>scratch</code><br/> Pointer to the scratch memory.<br/> 8 bytes alignment is required.</p> |
| <b>Note</b> | None                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

### Example

```
WORD16 inp_samples, out_bytes;
pWORD16 inp_speech;
pUWORD8 enc_speech;
xa_opus_enc_control_t enc_control;
pVOID scratch;

error_code = xa_opus_enc(
    xa_codec_handle_t handle,
    inp_speech,
    enc_speech,
    inp_samples,
    &enc_control,
    &out_bytes,
    scratch);
```

### Errors

- XA\_API\_FATAL\_MEM\_ALLOC
- XA\_API\_FATAL\_MEM\_ALIGN

- **XA\_OPUS\_EXECUTE\_FATAL\_NOT\_INITIALIZED**  
The Opus encoder is not yet initialized.
- **XA\_OPUS\_EXECUTE\_FATAL\_PARAMS\_CHANGED**  
Opus parameters changed. Opus must be reinitialized.
- **XA\_OPUS\_EXECUTE\_FATAL\_ENC\_INVALID\_NUM\_INP\_SAMPLES**  
Invalid (less than or equal to zero) number of input samples.
- **XA\_OPUS\_CONFIG\_FATAL\_ENC\_INVALID\_APP\_SETTING**  
The application mode is not supported.
- **XA\_OPUS\_CONFIG\_FATAL\_ENC\_INVALID\_BITRATE\_SETTING**  
Invalid bit rate.
- **XA\_OPUS\_CONFIG\_NONFATAL\_ENC\_INVALID\_FORCE\_CH\_SETTING**  
Invalid encoder channel force setting.
- **XA\_OPUS\_CONFIG\_NONFATAL\_ENC\_INVALID\_MAX\_BANDWIDTH\_SETTING**  
Invalid encoder max. bandwidth setting.
- **XA\_OPUS\_CONFIG\_NONFATAL\_ENC\_INVALID\_BANDWIDTH\_SETTING**  
Invalid encoder bandwidth setting.
- **XA\_OPUS\_CONFIG\_NONFATAL\_ENC\_INVALID\_COMPLEXITY\_SETTING**  
Invalid encoder complexity setting.
- **XA\_OPUS\_CONFIG\_NONFATAL\_ENC\_INVALID\_PACKET\_LOSS\_PERC\_SETTING**  
Invalid encoder expected packet loss percentage setting.
- **XA\_OPUS\_CONFIG\_NONFATAL\_ENC\_INVALID\_FORCE\_MODE\_SETTING**  
Invalid encoder force mode setting.
- **XA\_OPUS\_CONFIG\_NONFATAL\_ENC\_INVALID\_SIGNAL\_TYPE\_SETTING**  
Invalid encoder signal type setting.
- **XA\_OPUS\_EXECUTE\_NONFATAL\_INVALID\_FORCED\_MODE\_SETTINGS**  
Invalid force mode setting used.
- **XA\_OPUS\_EXECUTE\_NONFATAL\_BAD\_ARG**  
Invalid argument setting.
- **XA\_OPUS\_EXECUTE\_FATAL\_ENC\_OUT\_BUF\_TOO\_SHORT**  
The encoder output buffer is too short.
- **XA\_OPUS\_EXECUTE\_NONFATAL\_INTERNAL\_ERROR**  
Internal error.

## Opus Decoder's Process Function

Table 3-9 Opus Decoder's Process Function

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>    | <code>xa_opus_dec</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Syntax</b>      | <pre> XA_ERRORCODE xa_opus_dec (     xa_codec_handle_t handle,     pUWORD8 enc_speech,     pWORD16 synth_speech,     const WORD32 num_bytes_in,     xa_opus_dec_control_t *dec_control,     WORD16 *num_samples_out,     pVOID scratch); </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Description</b> | Decodes one frame of data.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Parameters</b>  | <p><b>Input:</b> <code>handle</code><br/> Pointer to the decoder handle (persistent state).<br/> 8 bytes alignment is required.</p> <p><b>Input:</b> <code>enc_speech</code><br/> Pointer to the encoded speech parameters (Opus core frame).<br/> 1 byte alignment is required.</p> <p><b>Output:</b> <code>synth_speech</code><br/> Pointer to the synthesized speech containing 16-bit PCM samples.<br/> 4 bytes alignment is required.</p> <p><b>Input:</b> <code>num_bytes_in</code><br/> Number of bytes in the input buffer.</p> <p><b>Input:</b> <code>dec_control</code><br/> Pointer to the decoder control structure.<br/> Refer to <i>xa_opus_dec_control_t</i> described in Section 3.3.</p> <p><b>Output:</b> <code>num_samples_out</code><br/> Number of output samples generated by the decoder.</p> <p><b>Input/Output:</b> <code>scratch</code><br/> Pointer to the scratch memory.<br/> 8 bytes alignment is required.</p> |
| <b>Note</b>        | None                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

## Example

```
WORD16 num_samples_out;
WORD32 num_bytes_in;
pUWORD8 enc_speech;
pWORD16 synth_speech;
xa_opus_dec_control_t dec_control;
pVOID scratch;

error_code = xa_opus_dec (
    xa_codec_handle_t handle,
    enc_speech,
    synth_speech,
    num_bytes_in,
    &dec_control,
    &num_samples_out,
    scratch);
```

## Errors

- XA\_API\_FATAL\_MEM\_ALLOC
- XA\_API\_FATAL\_MEM\_ALIGN
- XA\_OPUS\_EXECUTE\_FATAL\_NOT\_INITIALIZED  
The Opus decoder is not yet initialized.
- XA\_OPUS\_EXECUTE\_FATAL\_PARAMS\_CHANGED  
Opus parameters changed. Opus must be reinitialized.
- XA\_OPUS\_CONFIG\_NONFATAL\_DEC\_INVALID\_GAIN\_SETTING  
Invalid decoder gain adjustment setting.
- XA\_OPUS\_EXECUTE\_FATAL\_DEC\_PAYLOAD\_ERROR  
The compressed data passed is corrupted.
- XA\_OPUS\_EXECUTE\_NONFATAL\_BAD\_ARG  
Invalid arguments setting.
- XA\_OPUS\_EXECUTE\_NONFATAL\_INSUFFICIENT\_DATA  
Insufficient data.
- XA\_OPUS\_EXECUTE\_NONFATAL\_INTERNAL\_ERROR  
Internal error.
- XA\_OPUS\_EXECUTE\_NONFATAL\_INVALID\_PACKET  
Invalid packet.

- `XA_OPUS_EXECUTE_NONFATAL_UNSUPPORTED_MODE`  
Invalid Mode detected by the standalone SILK/CELT mode library.
- `XA_OPUS_EXECUTE_NONFATAL_HYBRID_MODE_DECODED`  
Hybrid Mode frame detected for standalone SILK mode library, Decoded frame with limited quality.

## 4. Introduction to the Example Testbench

The Opus Codec library is provided with two sample testbench applications: one for the encoder and the other for the decoder. The supplied test benches consist of the following files:

- Testbench source files (test/src):
  - `xa_opus_decoder_sample_testbench.c`
  - `xa_opus_encoder_sample_testbench.c`
  - `xa_opus_codec_error_handler.c`
  - `opus_header.c`
  - `xa_common_error_handler.c`
  - `xa_ogg_lib_error_handler.c`
- Makefile to build the executables (test/build):
  - `makefile_testbench_sample`

### 4.1 Making Executables

This section details how to build the testbench and the required libraries. You can build both the encoder and decoder testbench or a specific encoder or decoder testbench. Similarly, you can either build both libraries or a particular library.

## Building OPUS Applications

### Building Testbench Applications

- **Build Encoder and Decoder Testbench for Opus, Opus-SILK, and Opus-CELT**

1. In the command prompt, navigate to the `test/build` directory.
2. Enter the following command:  
`xt-make -f makefile_testbench_sample clean all`

The Opus encoder (`xa_opus_enc_test`) and the Opus decoder (`xa_opus_dec_test`) example test bench are built using the `xa_opus_codec.a` library.

The Opus-SILK encoder (`xa_opus_silk_enc_test`) and the Opus-SILK decoder (`xa_opus_silk_dec_test`) example test bench are built using the `xa_opus_silk_codec.a` library.

The Opus-CELT encoder (`xa_opus_celt_enc_test`) and the Opus-CELT decoder (`xa_opus_celt_dec_test`) example test bench are built using the `xa_opus_celt_codec.a` library.

### ▪ Build Either an Encoder or a Decoder Testbench for Opus, Opus-SILK, and Opus-CELT

1. In the command prompt, navigate to the `test/build` directory.

2. Enter the following command:

```
xt-make -f makefile_testbench_sample clean
[opus_dec|opus_silk_dec|opus_celt_dec|opus_enc|opus_silk_enc|opus_celt_
enc]
```

The encoder example testbench `xa_opus_enc_test/xa_opus_silk_enc_test/xa_opus_celt_enc_test` or the decoder example testbench `xa_opus_dec_test/xa_opus_silk_dec_test/xa_opus_celt_dec_test` is built.

## Building Libraries

### ▪ Build Libraries for Opus, Opus-SILK, and Opus-CELT

---

**Note** If you have source code distribution for the codec package, you must build the Opus codec libraries before you build the test bench.

---

1. In the command prompt, navigate to the `build` directory.

2. Enter the following command:

```
xt-make clean all install
```

The Opus codec libraries, namely `xa_opus_codec.a`, `xa_opus_enc.a`, `xa_opus_dec.a`, `xa_opus_silk_codec.a`, `xa_opus_silk_enc.a`, `xa_opus_silk_dec.a`, `xa_opus_celt_codec.a`, `xa_opus_celt_enc.a`, and `xa_opus_celt_dec.a` are built and copied to the `lib` directory..

### ▪ Build Either an Encoder or a Decoder Library for Opus, Opus-SILK, Opus-CELT

1. In the command prompt, navigate to the `build` directory.

2. Enter the following command:

```
xt-make clean [opus_dec install_dec|opus_silk_dec install_silk_dec|
opus_celt_dec install_celt_dec|opus_enc install_enc|opus_silk_enc
install_silk_enc|opus_celt_enc install_celt_enc]
```

The encoder `xa_opus_enc.a`, `xa_opus_silk_enc.a`, and `xa_opus_celt_enc.a` or the decoder `xa_opus_dec.a`, `xa_opus_silk_dec.a`, and `xa_opus_celt_dec.a` libraries are built and copied to the `lib` directory.

## XWS Package

---

**Note** For information on building and running the application from a Windows-based (.xws) based release package, refer to the *readme.html* file available in the imported application project.

---

In the XWS package, by default, the `testxa_opus_dec` decoder testbench project depends on the following:

- `ulibxa_opus_codec` library project in object .xws package.
- `libxa_opus_codec` library project in source .xws package.

## Switch Between Projects

To switch to the decoder ulibxa\_opus\_dec, perform the following steps in Xtensa Explorer:

1. In the **Project Explorer** area:
  - a. Click the triangle next to ulibxa\_opus\_dec to expand the folder.
  - b. Right-click ulibxa\_opus\_dec/lib/xa\_opus\_dec.a and select **Unmanaged Binary Info**.
  - c. Select the appropriate config (for example, AE\_HiFi3\_LE5) and click **OK**.
2. Right-click testxa\_opus\_dec and select **Library Dependencies....**  
ulibxa\_opus\_dec/lib/xa\_opus\_dec.a (AE\_HiFi3\_LE5) is shown in the Available libraries area.  
Select an item and click **Add**. The item appears in the **Selected libraries** area.
3. Select ulibxa\_opus\_codec/lib/xa\_opus\_codec.a (<none>) and click **Remove**, then click **Apply** and **OK**.

To switch to the decoder libxa\_opus\_dec, perform the following steps in Xtensa Explorer:

1. In the **Project Explorer** area, right-click testxa\_opus\_dec and select **Library Dependencies....**
2. In the **Available libraries** area, select libxa\_opus\_dec and click **Add**.  
The item appears in the Selected libraries area.
3. In the **Selected libraries** area, select libxa\_opus\_codec and click **Remove**, then click **Apply** and **OK**.

Similarly, the above steps can be used to switch to the encoder library projects ulibxa\_opus\_enc or libxa\_opus\_enc by replacing “dec” with “enc”.

The above steps are for Opus. Similarly, they can be used for Opus-SILK/Opus-CELT by replacing opus with opus\_silk/opus\_celt.

## 4.2 Usage

### 4.2.1 OPUS Usage

You can run the sample application executable using the direct command-line options or with a parameter file.

#### Encoder

You can run the sample application encoder executable using the command line as follows:

```
xt-run xa_opus_enc_test -app:<application> -fs:<sampling rate> -
numch:<number of channels> -bps:<bit rate> [options] -infile:<infile> -
ofile:<outfile>
```

Where:

|                       |                                                                                                                                                                                                                                                                         |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <application>:        | voip   audio   restricted-lowdelay                                                                                                                                                                                                                                      |
| <sampling rate>:      | 8000   12000   16000   24000   48000;                                                                                                                                                                                                                                   |
| <number of channels>: | 1-8; default: 2 (for raw opus streams)                                                                                                                                                                                                                                  |
| <bit rate>:           | Recommended target bit rate <6000..510000>, as described in Section 1.3; positive values outside the recommended range and -1 (maximum possible bitrate based on other parameters) are also accepted (this behavior is consistent with reference code <sup>[2]</sup> ). |
| <infile>:             | input speech file;                                                                                                                                                                                                                                                      |
| <outfile>:            | output encoded file;                                                                                                                                                                                                                                                    |
| [options]:            |                                                                                                                                                                                                                                                                         |
| -force_mode:          | <hybrid silk celt> forces the encoding mode; default: auto                                                                                                                                                                                                              |
| -cbr:                 | <0/1> enables constant bitrate; default: variable bitrate;                                                                                                                                                                                                              |
| -cvbr:                | <0/1> enables constrained variable bitrate; default: unconstrained;                                                                                                                                                                                                     |
| -bandwidth:           | <NB MB WB SWB FB> audio bandwidth (from narrowband to fullband); default: depends on sampling rate;                                                                                                                                                                     |
| -framesize:           | <2.5 5 10 20 40 60 80 100 120> frame size in ms; default: 20;                                                                                                                                                                                                           |
| -max_payload:         | <bytes> maximum payload size in bytes, (1-XA_OPUS_MAX_BYTES_ENCODED_PACKET-4); default: 1500;(Ignored if out of range);                                                                                                                                                 |
| -complexity:          | <comp> complexity, 0 (lowest) ... 10 (highest); default: 10;                                                                                                                                                                                                            |
| -inbandfec:           | <0/1> enables SILK inband FEC, default: disable;                                                                                                                                                                                                                        |

|                                 |                                                                                                                                                                                                                                           |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>-forcemono:</code>        | <0/1> forces mono encoding, even for stereo input, default: disable;                                                                                                                                                                      |
| <code>-dtx:</code>              | <0/1> enables SILK DTX, default: disable;                                                                                                                                                                                                 |
| <code>-loss:</code>             | <perc> uplinks loss estimate, in percent (0-100); default: 0;                                                                                                                                                                             |
| <code>-sweep:</code>            | <bps> sweeps bitrate mode, bps – bitrate step, default: 0, if non-zero, used as a step-size to change the bitrate, recommended bitrate range<6000..510000>;                                                                               |
| <code>-sweep_max:</code>        | <bps> maximum bitrate in sweep bitrate mode, default: 0;                                                                                                                                                                                  |
| <code>-random_framesize:</code> | <0/1> enables random frame size, default: disable;                                                                                                                                                                                        |
| <code>-random_fec:</code>       | <0/1> enables random SILK inband FEC, default: disable;                                                                                                                                                                                   |
| <code>-silk8k_test:</code>      | <0/1> enables dynamic changing of frame size and the number of channels in SILK mode (8 kHz) according to the predefined list, default: disable;                                                                                          |
| <code>-silk12k_test:</code>     | <0/1> enables dynamic changing of frame size and the number of channels in SILK mode (12 kHz) according to the predefined list, default: disable                                                                                          |
| <code>-silk16k_test:</code>     | <0/1> enables dynamic changing of frame size and the number of channels in SILK mode (16 kHz) according to the predefined list, default: disable;                                                                                         |
| <code>-hybrid24k_test:</code>   | <0/1> enables dynamic changing of frame size and the number of channels in Hybrid mode (24 kHz) according to the predefined list, default: disable;                                                                                       |
| <code>-hybrid48k_test:</code>   | <0/1> enables dynamic changing of frame size and the number of channels in Hybrid mode (48 kHz) according to the predefined list, default: disable;                                                                                       |
| <code>-celt_test:</code>        | <0/1> enables dynamic changing of frame size, number of channels, and bandwidth in CELT mode according to a predefined list, default: disable;                                                                                            |
| <code>-celt_hq_test:</code>     | <0/1> enables dynamic changing of frame size and the number of channels in CELT mode (FB only) according to the predefined list, default: disable.                                                                                        |
| <code>-btype:</code>            | <ogg/raw> bitstream type, ogg or raw; default: raw(mono/stereo), ogg(channels > 2).<br><br><b>Note:</b> This is a placeholder. Analysis is in progress to support raw mode for multi-stream if both encoder and decoder are from Cadence. |
| <code>-maxpage:n</code>         | Max Ogg page size: 'n' in kBytes, range: [1, 1024], default: 64;<br><br><b>Note:</b> This is a placeholder. The configurable page size option is not supported yet.                                                                       |
| <code>-maxdelay:n</code>        | Maximum container delay in milliseconds: [0, 1000], default: 1000;                                                                                                                                                                        |

`-force_mapping:` `<0/1/255>` forces the mapping family for multi-stream encoding; default: auto.

## Decoder

You can run the sample application decoder executable using the command line as follows:

```
xt-run xa_opus_dec_test [options] -ifile:<infile> -ofile:<outfile>
```

Where:

|                                  |                                                                                                                                                                                                                                                                                                                                 |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>&lt;infile&gt;:</code>     | input encoded speech file;                                                                                                                                                                                                                                                                                                      |
| <code>&lt;outfile&gt;:</code>    | output file with decoded samples;                                                                                                                                                                                                                                                                                               |
| <code>[options]:</code>          |                                                                                                                                                                                                                                                                                                                                 |
| <code>-fs:</code>                | 8000   12000   16000   24000   48000; default: 48000 (for raw opus streams)                                                                                                                                                                                                                                                     |
| <code>-numch:</code>             | 1-8; default: 2 (for raw opus streams)                                                                                                                                                                                                                                                                                          |
| <code>-inbandfec:</code>         | <code>&lt;0/1&gt;</code> enables SILK inband FEC (non-zero value is treated as 1), default: disable;                                                                                                                                                                                                                            |
| <code>-loss:</code>              | <code>&lt;perc&gt;</code> simulates packet loss, in percent (0-100); default: 0;                                                                                                                                                                                                                                                |
| <code>-btype:</code>             | <code>&lt;ogg/raw&gt;</code> bitstream type, ogg or raw; default: raw;                                                                                                                                                                                                                                                          |
| <code>-maxpage:n</code>          | Max Ogg page size: 'n' in kBytes, range: [1, 1024], default: 64;                                                                                                                                                                                                                                                                |
| <code>-strmap:&lt;map&gt;</code> | Output channel map, a string of max. 8 characters, range for each character 0-8. 0-7 to route the corresponding channel from decoder output, 8 for silence, for streams with less than 8 channels (n) range becomes 0-(n-1), default: taken from opus header. For interpretation of channels, refer to RFC7845 Section 5.1.1.2. |
| <code>-ch_mapping:</code>        | <code>&lt;0/1/255&gt;</code> Channel mapping family used for raw encoded streams; default: auto                                                                                                                                                                                                                                 |

---

**Note** The default values for the `-fs` and `-numch` options are applicable only for raw opus streams. For Ogg opus streams, `-fs` and `-numch` are taken from the ogg stream header without any check if not passed on the command line.

**Note** If the value for `-strmap` is specified partially, the remaining channels are silenced. If it is longer than the number of channels, then extra values are ignored. This option is ignored for raw opus streams and channel mapping family 0 ogg opus streams.

---

If command-line arguments are not provided, the encoder or decoder application reads the commands from the `paramfilesimple_encode.txt` or `paramfilesimple_decode.txt` parameter file, respectively.

The following is the syntax for writing the `paramfilesimple` file:

```
@Start

@Input_path <path to be appended to all input files>

@Output_path <path to be appended to all output files>
<command line 1>
<command line 2>
....
@Stop
```

The Opus encoder and decoder can be run for multiple test files using different command lines. The syntax for command lines in the parameter file is the same as the syntax for specifying options on the command line to the testbench program.

---

**Note** All the `@<command>`s should be at the first column of a line except the `@Newline` command.

**Note** All the `@<command>`s are case sensitive. If the command line in the parameter file has to be separated into two parts on two different lines, use the `@Newline` command.

For example,  
<command line part 1> @Newline  
<command line part 2>

**Note** Blank lines will be ignored.

**Note** Individual lines can be commented out using `"/"` at the beginning of the line.

**Note** The encoder's Ogg output in certain silk mode scenarios is not bit-exact with multichannel Ogg reference binary (opusenc - opus-tools-0.1.9). However, it is bit-exact in raw mode with opus encoder reference binary (opus\_demo - opus-1.3.1 ). Analysis is in progress.

---

## 4.2.2 OPUS-SILK Usage

You can run the sample application executable using the direct command-line options or with a parameter file.

### Encoder

You can run the sample application encoder executable using the command line as follows:

```
xt-run xa_opus_silk_enc_test -fs:<sampling rate> -numch:<channels> -
bps:<bit rate> [options] -infile:<input file> -ofile:<output file>
```

Where:

|                       |                                                                                                                                                                                                                                                                         |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <sampling rate>:      | 8000   12000   16000   24000   48000;                                                                                                                                                                                                                                   |
| <number of channels>: | 1   2; (Multichannel encoder is not supported)                                                                                                                                                                                                                          |
| <bit rate>:           | Recommended target bit rate <6000..510000>, as described in Section 1.3; positive values outside the recommended range and -1 (maximum possible bitrate based on other parameters) are also accepted (this behavior is consistent with reference code <sup>[2]</sup> ). |
| <infile>:             | input speech file;                                                                                                                                                                                                                                                      |
| <outfile>:            | output encoded file;                                                                                                                                                                                                                                                    |
| [options]:            |                                                                                                                                                                                                                                                                         |
| -cbr:                 | <0/1> enables constant bitrate; default: variable bitrate;                                                                                                                                                                                                              |
| -cvbr:                | <0/1> enables constrained variable bitrate; default: unconstrained;                                                                                                                                                                                                     |
| -bandwidth:           | <NB MB WB> audio bandwidth (from narrowband to wideband); default: sampling rate;                                                                                                                                                                                       |
| -framesize:           | <10 20 40 60 80 100 120> frame size in ms; default: 20;                                                                                                                                                                                                                 |
| -max_payload:         | <bytes> maximum payload size in bytes, (1-XA_OPUS_MAX_BYTES_ENCODED_PACKET-4); default: 1500;(Ignored if out of range);                                                                                                                                                 |
| -complexity:          | <comp> complexity, 0 (lowest) ... 10 (highest); default: 10;                                                                                                                                                                                                            |
| -inbandfec:           | <0/1> enables SILK inband FEC, default: disable;                                                                                                                                                                                                                        |

|                    |                                                                                                                                                                     |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -forcemono:        | <0/1> forces mono encoding, even for stereo input, default: disable;                                                                                                |
| -dtx:              | <0/1> enables SILK DTX, default: disable                                                                                                                            |
| -loss:             | <perc> uplinks loss estimate, in percent (0-100); default: 0;                                                                                                       |
| -sweep:            | <bps> sweeps bitrate mode, bps – bitrate step, default: 0, if non-zero, used as a step-size to change the bitrate, recommended bitrate range<6000..510000>;         |
| -sweep_max:        | <bps> maximum bitrate in sweep bitrate mode, default: 0;                                                                                                            |
| -random_framesize: | <0/1> enables random frame size, default: disable;                                                                                                                  |
| -random_fec:       | <0/1> enables random SILK inband FEC, default: disable;                                                                                                             |
| -silk8k_test:      | <0/1> enables dynamic changing of frame size and the number of channels in SILK mode (8 kHz) according to the predefined list, default: disable;                    |
| -silk12k_test:     | <0/1> enables dynamic changing of frame size and the number of channels in SILK mode (12 kHz) according to the predefined list, default: disable                    |
| -silk16k_test:     | <0/1> enables dynamic changing of frame size and the number of channels in SILK mode (16 kHz) according to the predefined list, default: disable;                   |
| -bytppe:           | <ogg/raw> bitstream type, ogg or raw; default: raw;                                                                                                                 |
| -maxpage:n         | Max Ogg page size: 'n' in kBytes, range: [1, 1024], default: 64;<br><br><b>Note:</b> This is a placeholder. The configurable page size option is not supported yet. |
| -maxdelay:n        | Maximum container delay in milliseconds: [0, 1000], default: 1000;                                                                                                  |

## Decoder

You can run the sample application decoder executable using the command line as follows:

```
xt-run xa_opus_silk_dec_test -fs:<sampling_rate> -numch:<channels>
[options] -ifile:<input_file> -ofile:<output_file>
```

Where:

|                |                                                                                                                                                                                                                                                                                                                               |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input_file>:  | input encoded speech file;                                                                                                                                                                                                                                                                                                    |
| <output_file>: | output file with decoded samples;                                                                                                                                                                                                                                                                                             |
| [options]:     |                                                                                                                                                                                                                                                                                                                               |
| -fs:           | 8000   12000   16000   24000   48000; default: 48000 (for raw opus streams)                                                                                                                                                                                                                                                   |
| -numch:        | 1-8; default: 2 (for raw opus streams)                                                                                                                                                                                                                                                                                        |
| -inbandfec:    | <0/1 enables SILK inband FEC (non-zero value is treated as 1), default: disable;                                                                                                                                                                                                                                              |
| -loss:         | <perc> simulates packet loss, in percent (0-100); default: 0;                                                                                                                                                                                                                                                                 |
| -btype:        | <ogg/raw> bitstream type, ogg or raw; default: raw;                                                                                                                                                                                                                                                                           |
| -maxpage:n     | Max Ogg page size: 'n' in kBytes, range: [1, 1024], default: 64;                                                                                                                                                                                                                                                              |
| -strmap:<map>  | Output channel map, string of max. 8 characters, range for each character 0-8. 0-7 to route the corresponding channel from decoder output, 8 for silence, for streams with less than 8 channels (n) range becomes 0-(n-1), default: taken from opus header. For interpretation of channels, refer to RFC7845 Section 5.1.1.2. |

---

**Note** The default values for the `-fs` and `-numch` options are applicable only for raw opus streams. For Ogg opus streams, `-fs` and `-numch` are taken from the ogg stream header without any check if not passed on the command line.

**Note** If the value for `-strmap` is specified partially, remaining channels are silenced. If it is longer than the number of channels, then extra values are ignored. This option is ignored for raw opus streams and channel mapping family 0 ogg opus streams.

---

If command-line arguments are not provided, the encoder or decoder application reads the commands from the `paramfilesimple_encode_silk.txt` or `paramfilesimple_decode_silk.txt` parameter file, respectively.

## 4.2.3 OPUS-CELT Usage

You can run the sample application executable using the direct command-line options or with a parameter file.

### Encoder

You can run the sample application encoder executable using the command line as follows:

```
xt-run xa_opus_celt_enc_test -app:<application> -fs:<sampling rate> -
numch:<channels> -bps:<bit rate> [options] -ifile:<input file> -
ofile:<output file>
```

Where:

|                       |                                                                                                                                                                                                                                                                         |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <application>:        | audio   restricted-lowdelay                                                                                                                                                                                                                                             |
| <sampling rate>:      | 8000   12000   16000   24000   48000;                                                                                                                                                                                                                                   |
| <number of channels>: | 1-8; default: 2 (for raw opus streams)                                                                                                                                                                                                                                  |
| <bit rate>:           | Recommended target bit rate <6000..510000> as described in Section 1.3; positive values outside the recommended range and -1 (maximum possible bitrate based on other parameters) are also accepted (this behavior is consistent with reference code <sup>[21]</sup> ). |
| <input file>:         | input speech file;                                                                                                                                                                                                                                                      |
| <output file>:        | output encoded file;                                                                                                                                                                                                                                                    |
| [options]:            |                                                                                                                                                                                                                                                                         |
| -cbr:                 | <0/1> enables constant bitrate; default: variable bitrate;                                                                                                                                                                                                              |
| -cvbr:                | <0/1> enables constrained variable bitrate; default: unconstrained;                                                                                                                                                                                                     |
| -bandwidth:           | < NB MB WB SWB FB > audio bandwidth (from narrowband to fullband); default: sampling rate;                                                                                                                                                                              |
| -framesize:           | < 2.5 5 10 20 40 60 80 100 120> frame size in ms; default: 20;                                                                                                                                                                                                          |
| -max_payload:         | <bytes> maximum payload size in bytes, (1-XA_OPUS_MAX_BYTES_ENCODED_PACKET-4); default: 1500; (Ignored if out of range);                                                                                                                                                |
| -complexity:          | <comp> complexity, 0 (lowest) ... 10 (highest); default: 10;                                                                                                                                                                                                            |

|                                 |                                                                                                                                                                                                                                                              |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>-forcemono:</code>        | <code>&lt;0/1&gt;</code> forces mono encoding, even for stereo input, default: disable;                                                                                                                                                                      |
| <code>-loss:</code>             | <code>&lt;perc&gt;</code> uplinks loss estimate, in percent (0-100); default: 0;                                                                                                                                                                             |
| <code>-sweep:</code>            | <code>&lt;bps&gt;</code> sweeps bitrate mode, bps – bitrate step, default: 0, if non-zero, used as a step-size to change the bitrate, recommended bitrate range <code>&lt;6000..510000&gt;</code> ;                                                          |
| <code>-sweep_max:</code>        | <code>&lt;bps&gt;</code> maximum bitrate in sweep bitrate mode, default: 0;                                                                                                                                                                                  |
| <code>-random_framesize:</code> | <code>&lt;0/1&gt;</code> enables random frame size, default: disable;                                                                                                                                                                                        |
| <code>-celt_test:</code>        | <code>&lt;0/1&gt;</code> enables dynamic changing of frame size, number of channels, and bandwidth in CELT mode according to the predefined list, default: disable;                                                                                          |
| <code>-celt_hq_test:</code>     | <code>&lt;0/1&gt;</code> enables dynamic changing of frame size and the number of channels in CELT mode (FB only) according to the predefined list, default: disable.                                                                                        |
| <code>-btype:</code>            | <code>&lt;ogg/raw&gt;</code> bitstream type, ogg or raw; default: raw(mono/stereo), ogg(channels > 2).<br><br><b>Note:</b> This is a placeholder. Analysis is in progress to support raw mode for multi-stream if both encoder and decoder are from Cadence. |
| <code>-maxpage:n</code>         | Max Ogg page size: 'n' in kBytes, range: [1, 1024], default: 64;<br><br><b>Note:</b> This is a placeholder. The configurable page size option is not supported yet.                                                                                          |
| <code>-maxdelay:n</code>        | Maximum container delay in milliseconds: [0, 1000], default: 1000;                                                                                                                                                                                           |
| <code>-force_mapping:</code>    | <code>&lt;0/1/255&gt;</code> forces the mapping family for multi-stream encoding; default: auto.                                                                                                                                                             |

## Decoder

You can run the sample application decoder executable using the command line as follows:

```
xt-run xa_opus_celt_dec_test -fs:<sampling rate> -numch:<channels>
[options] -ifile:<input_file> -ofile:<output_file>
```

Where:

|                |                                                                                                                                                                                                                                                                                                                               |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input_file>:  | input encoded speech file;                                                                                                                                                                                                                                                                                                    |
| <output_file>: | output file with decoded samples;                                                                                                                                                                                                                                                                                             |
| [options]:     |                                                                                                                                                                                                                                                                                                                               |
| -fs:           | 8000   12000   16000   24000   48000; default: 48000 (for raw opus streams)                                                                                                                                                                                                                                                   |
| -numch:        | 1-8; default: 2 (for raw opus streams)                                                                                                                                                                                                                                                                                        |
| -loss:         | <perc> simulates packet loss, in percent (0-100); default: 0;                                                                                                                                                                                                                                                                 |
| -btype:        | <ogg/raw> bitstream type, ogg or raw; default: raw;                                                                                                                                                                                                                                                                           |
| -maxpage:n     | Max Ogg page size: 'n' in kBytes, range: [1, 1024], default: 64;                                                                                                                                                                                                                                                              |
| -strmap:<map>  | Output channel map, string of max. 8 characters, range for each character 0-8. 0-7 to route the corresponding channel from decoder output, 8 for silence, for streams with less than 8 channels (n) range becomes 0-(n-1), default: taken from opus header. For interpretation of channels, refer to RFC7845 Section 5.1.1.2. |
| -ch_mapping:   | <0/1/255> Channel mapping family used for raw encoded streams; default: auto                                                                                                                                                                                                                                                  |

**Note** The default values for the `-fs` and `-numch` options are applicable only for raw opus streams. For Ogg opus streams, `-fs` and `-numch` are taken from the ogg stream header without any check if not passed on the command line.

**Note** If the value for `-strmap` is specified partially, remaining channels are silenced, if it is longer than the number of channels, then extra values are ignored. This option is ignored for raw opus streams and channel mapping family 0 ogg opus streams.

If command-line arguments are not provided, the encoder or decoder application reads the commands from the `paramfilesimple_encode_celt.txt` or `paramfilesimple_decode_celt.txt` parameter file, respectively.

## 5. Appendix – Ogg Container Format Support in Encoder and Decoder

---

The HiFi Opus decoder supports encoding and decoding of Opus streams in Ogg container format. The Ogg container format support is provided through a separate HiFi Ogg Packer and Parser interface. This HiFi Ogg Packer/Parser implements the Ogg encapsulation standard defined in RFC 3533 <sup>[3]</sup> and RFC 7845 <sup>[4]</sup> and is based on libogg 1.3.2 <sup>[5]</sup>.

The HiFi Ogg Packer/Parser performance is mentioned in Section 1.4. The following sections briefly describe the HiFi Ogg Packer/Parser details.

### 5.1 Ogg Packer/Parser-specific Files

The Ogg Packer/Parser parameter header file (`include/ogg_lib`):

- `xa_ogg_lib_api.h`

### 5.2 I/O Formats

The HiFi Ogg Packer/Parser does not put any restriction on the alignment and size of the input buffer. Smaller input buffer sizes may require multiple calls to the Ogg Parser process function to parse one Ogg page. The testbench application allocates a 4 kBytes input buffer.

The Ogg packer writes the output buffer only after one ogg page is available. The default ogg page size is 64KB. A configurable page size for the packer is not supported yet.

The HiFi Ogg Packer/Parser does not put any restriction on the alignment and size of the output buffer.

---

|             |                                                                                                                                                                           |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Note</b> | If the output buffer is not sufficient to hold the produced Opus packet, the HiFi Ogg Parser will return a fatal error indicating the required size of the output buffer. |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

For more information, see Section 5.4.3. The testbench application uses a 3 kBytes output buffer to hold two encoded Opus packets.

## 5.3 Configuration Structure

Table 5-1 xa\_ogg\_pack\_init\_cfg\_t Structure Parameters

| Parameter Name | Value Type | Range / Supported Values             | Default | Description                                                                                                          |
|----------------|------------|--------------------------------------|---------|----------------------------------------------------------------------------------------------------------------------|
| input_Fs_fx    | WORD32     | 8000   12000   16000   24000   48000 | 48000   | Input sampling rate                                                                                                  |
| frame_size     | WORD32     | Supported by Opus Lib                | 960     | Input frame size in samples                                                                                          |
| max_ogg_delay  | WORD32     | 0-1000                               | 1000    | Maximum container delay in milliseconds                                                                              |
| serialno       | WORD32     | -                                    | -       | Specific stream serial number                                                                                        |
| max_page_size  | WORD32     | 1 to 1024                            | 64      | Maximum page size to be allocated in Kbytes. Values outside the range are ignored, and default is used in that case. |
| skip           | WORD32     | -                                    | -       | Skip bytes to be used for delay compensation                                                                         |

Table 5-2 xa\_ogg\_parse\_init\_cfg\_t Structure Parameters

| Parameter Name | Value Type | Range / Supported Values | Default | Description                                                                                                          |
|----------------|------------|--------------------------|---------|----------------------------------------------------------------------------------------------------------------------|
| max_page_size  | WORD32     | 1 to 1024                | 64      | Maximum page size to be allocated in Kbytes. Values outside the range are ignored, and default is used in that case. |

## 5.4 API Functions

The Ogg Packer/Parser API functions relevant to each stage in the codec flow are specified in the following sections.

### 5.4.1 Memory Allocation Stage

During the memory allocation stage, the application needs to reserve the necessary memory for the HiFi Ogg Packer/Parser handles (persistent state) and scratch buffers. The required alignment of the handles and the scratch buffers is eight bytes. The application can use the functions listed in Table 5-3 to query the Ogg Packer/Parser for the required size of each buffer. The functions take a pointer of type `xa_ogg_pack_init_cfg_t/xa_ogg_parse_init_cfg_t` and return `WORD32`. `xa_ogg_pack_init_cfg_t/xa_ogg_parse_init_cfg_t` is described in Table 5-4 and Table 5-5.

`xa_ogg_pack_init_cfg_t/xa_ogg_parse_init_cfg_t` is a structure that contains the initialization parameters for this instance of the Ogg Packer/Parser. The default initial configuration parameters are used if `NULL` is passed instead of a valid pointer.

The input and output frame buffers are required for the component's operation. However, these buffers do not need to be reserved at this stage. Instead, pointers to the frame buffers are passed in each invocation of the codec's process function.

The size and alignment requirements of the I/O buffers are specified in Section 5.2.

Table 5-3 Memory Management Functions

| Function                                        | Description                                                                |
|-------------------------------------------------|----------------------------------------------------------------------------|
| <code>xa_ogg_pack_get_handle_byte_size</code>   | Returns the size of the Ogg Packer API handle (persistent state) in bytes. |
| <code>xa_ogg_pack_get_scratch_byte_size</code>  | Returns the size of the Ogg Packer scratch memory.                         |
| <code>xa_ogg_parse_get_handle_byte_size</code>  | Returns the size of the Ogg Parser API handle (persistent state) in bytes. |
| <code>xa_ogg_parse_get_scratch_byte_size</code> | Returns the size of the Ogg Parser scratch memory.                         |

## Example

```
WORD32 handle_size;
WORD32 scratch_size;
handle_size = xa_ogg_pack_get_handle_byte_size(NULL);
scratch_size = xa_ogg_pack_get_scratch_byte_size(NULL);

handle_size = xa_ogg_parse_get_handle_byte_size(NULL);
scratch_size = xa_ogg_parse_get_scratch_byte_size(NULL);
```

## Errors

- None

## 5.4.2 Initialization Stage

In the initialization stage, the application points the HiFi Ogg Packer/Parser to its API handle and scratch buffer. The application also specifies various other parameters related to the Ogg Packer's/Parser's operation and places the Ogg Packer/Parser in its initial state. The API functions for the HiFi Ogg Packer/Parser initialization are specified in Table 5-4 and Table 5-5.

Table 5-4 HiFi Ogg Pack Initialization Function Details

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>    | <code>xa_ogg_pack_init</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Syntax</b>      | <pre>XA_ERRORCODE xa_ogg_pack_init (     xa_codec_handle_t handle,     pVOID scratch,     xa_ogg_parse_init_cfg_t *cfg)</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Description</b> | Resets the HiFi Ogg Packer API handle to its initial state. Sets up the component to run using the supplied scratch buffer and the specified initial configuration parameters.                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Parameters</b>  | <p><b>Input:</b> <code>handle</code><br/> Pointer to the component persistent memory. This is an opaque handle.<br/> Required size: See <code>xa_ogg_pack_get_handle_byte_size</code><br/> Required alignment: 8 bytes</p> <p><b>Input:</b> <code>scratch</code><br/> Pointer to the component scratch buffer<br/> Required size: See <code>xa_ogg_pack_get_scratch_byte_size</code><br/> Required alignment: 8 bytes</p> <p><b>Input:</b> <code>p_cfg</code><br/> Initial configuration parameters (see Section 5.3).</p> <p><b>Note:</b> The initial configuration parameters MUST be identical to those passed during the memory allocation stage.</p> |

### Example

```
xa_codec_handle_t handle =
    (xa_codec_handle_t)malloc(handle_size);
pVOID scratch = malloc(scratch_size);
res = xa_ogg_pack_init(handle, scratch, NULL);
```

### Errors

- `XA_API_FATAL_MEM_ALLOC`  
`handle` or `scratch` is `NULL`.
- `XA_API_FATAL_MEM_ALIGN`

`handle` or `scratch` is not aligned correctly.

- **XA\_OGG\_CONFIG\_FATAL\_INIT\_FAILED**  
Ogg library initialization failed.
- **XA\_OGG\_CONFIG\_FATAL\_SAMP\_FREQ\_NOT\_SUPPORTED**  
The sampling frequency is not supported.

Table 5-5 HiFi Ogg Parse Initialization Function Details

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>    | <code>xa_ogg_parse_init</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Syntax</b>      | <pre>XA_ERRORCODE xa_ogg_parse_init (     xa_codec_handle_t handle,     pVOID scratch,     xa_ogg_parse_init_cfg_t *cfg)</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Description</b> | Resets the HiFi Ogg Parser API handle to its initial state. Sets up the component to run using the supplied scratch buffer and the specified initial configuration parameters.                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Parameters</b>  | <p><b>Input:</b> <code>handle</code><br/>Pointer to the component persistent memory. This is an opaque handle.<br/>Required size: See <code>xa_ogg_parse_get_handle_byte_size</code><br/>Required alignment: 8 bytes</p> <p><b>Input:</b> <code>scratch</code><br/>Pointer to the component scratch buffer.<br/>Required size: See <code>xa_ogg_parse_get_scratch_byte_size</code><br/>Required alignment: 8 bytes</p> <p><b>Input:</b> <code>p_cfg</code><br/>Initial configuration parameters (see Section 5.3).</p> <p><b>Note:</b> The initial configuration parameters MUST be identical to those passed during the memory allocation stage.</p> |

### Example

```
xa_codec_handle_t handle =
    (xa_codec_handle_t)malloc(handle_size);
pVOID scratch = malloc(scratch_size);
res = xa_ogg_parse_init(handle, scratch, NULL);
```

### Errors

- **XA\_API\_FATAL\_MEM\_ALLOC**  
`handle` or `scratch` is NULL.
- **XA\_API\_FATAL\_MEM\_ALIGN**

`handle` or `scratch` is not aligned correctly.

### 5.4.3 Process Stage

The HiFi Ogg Packer processes the Opus encoded stream frame-by-frame and generates the output pages. At each call, Opus encoded data is provided to the component's processing function, and if sufficient input data is provided, it is packed in the Ogg page as output. If partial output data is available, it is buffered internally and provided only when a complete Ogg page is available. The application fills the input buffer with the next input data and calls the processing function again.

The HiFi Ogg Parser processes the input stream and generates the output stream frame-by-frame. Each call to the component's processing function produces one complete Opus packet as output if sufficient input data is provided. If partial input data is presented, it is buffered internally, and the need-more-data is signaled to the application through the proper error code. The application fills the input buffer with the next input data and calls the processing function again.

The codec uses default parameters to perform the processing. These parameters can be changed or queried at any time after the initialization stage.

Table 5-6 Process Stage Functions

| Function                            | Description                                  |
|-------------------------------------|----------------------------------------------|
| <code>xa_ogg_pack_process</code>    | Processes one frame of encoded data.         |
| <code>xa_ogg_pack_set_param</code>  | Sets the value of a particular parameter.    |
| <code>xa_ogg_pack_get_param</code>  | Returns the value of a particular parameter. |
| <code>xa_ogg_parse_process</code>   | Processes one frame of audio data.           |
| <code>xa_ogg_parse_set_param</code> | Sets the value of a particular parameter.    |
| <code>xa_ogg_parse_get_param</code> | Returns the value of a particular parameter. |

The syntax of the execution stage functions is specified in the following tables.

#### HiFi Ogg Packer/Parser Process Functions

Table 5-7 HiFi Ogg Packer Process Function Details

|                    |                                                                                                                                                                            |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>    | <code>xa_ogg_pack_process</code>                                                                                                                                           |
| <b>Syntax</b>      | <pre>XA_ERRORCODE xa_ogg_pack_process (     xa_codec_handle_t handle,     pUWORD8 in_data,     pUWORD8 out_data,     WORD32 num_bytes_in,     pWORD32 num_bytes_out)</pre> |
| <b>Description</b> | Processes one frame of encoded speech using the current component state.                                                                                                   |

|                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Parameters</b> | <p><b>Input:</b> <code>handle</code><br/>Opaque component handle.</p> <p><b>Input:</b> <code>in_data</code><br/>A pointer to the input encoded speech from which the input data will be read. This is the input buffer.<br/>Required alignment: 1 byte</p> <p><b>Output:</b> <code>out_data</code><br/>A pointer to the output page into which the output data will be written. This is the output buffer.<br/>Required alignment: 1 byte</p> <p><b>Input/Output:</b> <code>num_bytes_in</code><br/>Pointer to the number of input bytes. This contains the amount of data to be processed.</p> <p><b>Input/Output:</b> <code>num_bytes_out</code><br/>Pointer to the number of output bytes. This contains the space available in the output buffer in terms of bytes. On return, <code>*num_bytes_out</code> is set to the actual number of output bytes generated by the codec.</p> |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Example

```

UWORD8 p_in_data[512];
UWORD8 p_out_data[64*1024];
WORD32 in_samples = 512;
WORD32 out_samples = 0;
res = xa_ogg_pack_process(handle, p_in_data, p_out_data,
                          in_samples, &out_samples);

```

### Errors

- **XA\_API\_FATAL\_MEM\_ALLOC**  
One of the pointers (`handle`, `p_in_data`, `p_out_data`, or `p_out_samples`) is NULL.
- **XA\_API\_FATAL\_MEM\_ALIGN**  
The codec handle is not aligned correctly.
- **XA\_OGG\_EXECUTE\_FATAL\_NOT\_INITIALIZED**  
Ogg Packer is not initialized.
- **XA\_OGG\_EXECUTE\_FATAL\_PAGE\_SIZE\_TOO\_SMALL**  
The maximum page size allocated is too small to fit one page for the current input stream.  
The application should re-initialize Ogg Packer with a higher page size.

Table 5-8 HiFi Ogg Parser Process Function Details

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>    | xa_ogg_parse_process                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Syntax</b>      | <p>XA_ERRORCODE</p> <p>xa_ogg_parse_process (</p> <p style="padding-left: 40px;">xa_codec_handle_t handle,</p> <p style="padding-left: 40px;">pUWORD8 in_data,</p> <p style="padding-left: 40px;">pUWORD8 out_data,</p> <p style="padding-left: 40px;">pWORD32 num_bytes_in,</p> <p style="padding-left: 40px;">pWORD32 num_bytes_out)</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Description</b> | Processes one frame of audio using the current component state.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Parameters</b>  | <p><b>Input:</b> handle</p> <p>Opaque component handle.</p><br><p><b>Input:</b> in_data</p> <p>A pointer to the input audio stream from which the input data will be read. This is the input buffer.</p> <p>Required alignment: 1 byte</p><br><p><b>Output:</b> out_data</p> <p>A pointer to the output audio stream into which the output data will be written. This is the output buffer.</p> <p>Required alignment: 1 byte</p><br><p><b>Input/Output:</b> num_bytes_in</p> <p>Pointer to the number of input bytes. This contains the amount of data to be processed. On return, *num_bytes_in is set to the actual amount of data processed.</p><br><p><b>Input/Output:</b> num_bytes_out</p> <p>Pointer to the number of output bytes. This contains the space available in the output buffer in terms of bytes. On return, *num_bytes_out is set to the actual number of output bytes generated by the codec. In case if the output space is smaller than the output bytes generated, *num_bytes_out will indicate the new output space required.</p> |

### Example

```

UWORD8 p_in_data[512];
UWORD8 p_out_data[512];
WORD32 in_samples = 512;
WORD32 out_samples = 512;
res = xa_ogg_parse_process(handle, p_in_data, p_out_data,
                           &in_samples, &out_samples);

```

## Errors

- **XA\_API\_FATAL\_MEM\_ALLOC**  
One of the pointers (`handle`, `p_in_data`, `p_out_data`, `p_in_samples`, or `p_out_samples`) is NULL.
- **XA\_API\_FATAL\_MEM\_ALIGN**  
The codec handle is not aligned correctly.
- **XA\_OGG\_EXECUTE\_FATAL\_NOT\_INITIALIZED**  
Ogg Parser is not initialized.
- **XA\_OGG\_EXECUTE\_FATAL\_PAGE\_SIZE\_TOO\_SMALL**  
The maximum page size allocated is too small to fit one page for the current input stream.  
The application should re-initialize Ogg Parser with a higher page size.
- **XA\_OGG\_EXECUTE\_FATAL\_CORRUPTED\_STREAM**  
Corrupted input stream, two streams with the same Opus serial number detected in the input stream.
- **XA\_OGG\_EXECUTE\_FATAL\_EXTRA\_PKTS\_ON\_HEADER\_PAGE**  
Corrupted input stream, extra packets detected on the header page.
- **XA\_OGG\_EXECUTE\_FATAL\_EXTRA\_PKTS\_ON\_TAGS\_PAGE**  
Corrupted input stream, extra packets detected on the tags page.
- **XA\_OGG\_EXECUTE\_FATAL\_OUT\_BUF\_TOO\_SMALL**  
Output buffer space is too small for output generated for the current input stream.  
The application should reinitialize Ogg Parser with a higher output buffer size; `*num_bytes_out` should indicate the new required size in this case.
- **XA\_OGG\_EXECUTE\_NONFATAL\_INSUFFICIENT\_DATA**  
Insufficient input data.  
The application should call the process function again with the new input data.
- **XA\_OGG\_EXECUTE\_NONFATAL\_STREAM\_CHANGE**  
A new Opus stream (Opus header) is found before the end-of-stream for the current stream.  
The application should treat the current output packet as an Opus header and the next output packet as a tags packet and process them accordingly. The Opus decoder should be reinitialized with the new stream parameters parsed above.
- **XA\_OGG\_EXECUTE\_NONFATAL\_PAGE\_IGNORED**  
Spurious pages ignored (for example, pages before the Opus header page, or pages with different serial numbers than the one currently being decoded).

## HiFi Ogg Packer/Parser Set Parameter Function

Table 5-9 HiFi Ogg Packer Set Parameter Function Details

|                    |                                                                                                                                                                                                                                                                                                                                  |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>    | xa_ogg_pack_set_param                                                                                                                                                                                                                                                                                                            |
| <b>Syntax</b>      | XA_ERRORCODE<br>xa_ogg_pack_set_param (<br>xa_codec_handle_t handle,<br>xa_ogg_pack_param_id_t param_id,<br>pVOID p_param_value)<br>                                                                                                                                                                                             |
| <b>Description</b> | Sets the parameter specified by param_id to the value passed in the buffer pointed to by p_param_value.                                                                                                                                                                                                                          |
| <b>Parameters</b>  | <p><b>Input:</b> handle<br/>Opaque component handle.</p> <p><b>Input:</b> param_id<br/>Identifies the parameter to be written. For the list of supported parameters, refer to Section 5.4.4.</p> <p><b>Input:</b> p_param_value<br/>A pointer to a buffer that contains the parameter value.<br/>Required alignment: 4 bytes</p> |

### Example

```
WORD32 param_id = XA_OGG_PACK_PARAM_END_OF_STREAM;
WORD32 eos;
res = xa_ogg_pack_set_param(handle, param_id, &eos);
```

### Errors

- **XA\_API\_FATAL\_MEM\_ALLOC**  
One of the pointers (handle or p\_param\_value) is NULL
- **XA\_API\_FATAL\_MEM\_ALIGN**  
One of the pointers (handle or p\_param\_value) is not aligned correctly.
- **XA\_OGG\_EXECUTE\_FATAL\_NOT\_INITIALIZED**  
The function is called before HiFi Ogg Packer is initialized.
- **XA\_OGG\_CONFIG\_NONFATAL\_INVALID\_PARAM**  
param\_id is not valid.

Table 5-10 HiFi Ogg Parser Set Parameter Function Details

|                    |                                                                                                                                                                                                                                                                                                                                                                          |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>    | <code>xa_ogg_parse_set_param</code>                                                                                                                                                                                                                                                                                                                                      |
| <b>Syntax</b>      | <code>XA_ERRORCODE</code><br><code>xa_ogg_parse_set_param (</code><br><code>xa_codec_handle_t handle,</code><br><code>xa_ogg_parse_param_id_t param_id,</code><br><code>pVOID p_param_value)</code>                                                                                                                                                                      |
| <b>Description</b> | <p>Sets the parameter specified by <code>param_id</code> to the value passed in the buffer pointed to by <code>p_param_value</code>.</p> <p><b>Note:</b> This function is provided only for API completeness. No config parameter is to be set.</p>                                                                                                                      |
| <b>Parameters</b>  | <p><b>Input:</b> <code>handle</code><br/>Opaque component handle.</p> <p><b>Input:</b> <code>param_id</code><br/>Identifies the parameter to be written. No parameters are supported for this function in this version.</p> <p><b>Input:</b> <code>p_param_value</code><br/>A pointer to a buffer that contains the parameter value.<br/>Required alignment: 4 bytes</p> |

## Errors

- `XA_API_FATAL_MEM_ALLOC`  
One of the pointers (`handle` or `p_param_value`) is NULL
- `XA_API_FATAL_MEM_ALIGN`  
One of the pointers (`handle` or `p_param_value`) is not aligned correctly.
- `XA_OGG_EXECUTE_FATAL_NOT_INITIALIZED`  
The function is called before HiFi Ogg Parser is initialized.
- `XA_OGG_CONFIG_NONFATAL_INVALID_PARAM`  
`param_id` is not valid.

## HiFi Ogg Packer/Parser Get Parameter Function

Table 5-11 HiFi Ogg Packer Get Parameter Function Details

|                    |                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>    | <code>xa_ogg_pack_get_param</code>                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Syntax</b>      | <code>XA_ERRORCODE</code><br><code>xa_ogg_pack_get_param (</code><br><code>xa_codec_handle_t handle,</code><br><code>xa_ogg_pack_param_id_t param_id,</code><br><code>pVOID p_param_value)</code>                                                                                                                                                                                                     |
| <b>Description</b> | Gets the parameter value specified by <code>param_id</code> in the buffer pointed to by <code>p_param_value</code> .                                                                                                                                                                                                                                                                                  |
| <b>Parameters</b>  | <p><b>Input:</b> <code>handle</code><br/>Opaque component handle.</p> <p><b>Input:</b> <code>param_id</code><br/>Identifies the parameter to be read. For the list of supported parameters, refer to Section 5.4.4.</p> <p><b>Output:</b> <code>p_param_value</code><br/>A pointer to a buffer that is filled with the parameter value when the function returns.<br/>Required alignment: 4 bytes</p> |

### Example

```
WORD32 param_id = XA_OGG_PACK_PARAM_END_OF_STREAM;
WORD32 eos;
res = xa_ogg_pack_get_param(handle, param_id, &eos);
```

### Errors

- **XA\_API\_FATAL\_MEM\_ALLOC**  
One of the pointers (`handle` or `p_param_value`) is `NULL`.
- **XA\_API\_FATAL\_MEM\_ALIGN**  
One of the pointers (`handle` or `p_param_value`) is not aligned correctly.
- **XA\_OGG\_EXECUTE\_FATAL\_NOT\_INITIALIZED**  
The function is called before HiFi Ogg Packer is initialized.

Table 5-12 HiFi Ogg Parser Get Parameter Function Details

|                    |                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>    | <code>xa_ogg_parse_get_param</code>                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Syntax</b>      | <code>XA_ERRORCODE</code><br><code>xa_ogg_parse_get_param (</code><br><code>xa_codec_handle_t handle,</code><br><code>xa_ogg_parse_param_id_t param_id,</code><br><code>pVOID p_param_value)</code>                                                                                                                                                                                                   |
| <b>Description</b> | Gets the parameter value specified by <code>param_id</code> in the buffer pointed to by <code>p_param_value</code>                                                                                                                                                                                                                                                                                    |
| <b>Parameters</b>  | <p><b>Input:</b> <code>handle</code><br/>Opaque component handle.</p> <p><b>Input:</b> <code>param_id</code><br/>Identifies the parameter to be read. For the list of supported parameters, refer to Section 5.4.4.</p> <p><b>Output:</b> <code>p_param_value</code><br/>A pointer to a buffer that is filled with the parameter value when the function returns.<br/>Required alignment: 4 bytes</p> |

### Example

```
WORD32 param_id = XA_OGG_PARSE_PARAM_PAGE_GRANULE;
WORD64 granule_pos;
res = xa_ogg_parse_get_param(handle, param_id, &granule_pos);
```

### Errors

- **XA\_API\_FATAL\_MEM\_ALLOC**  
One of the pointers (`handle` or `p_param_value`) is `NULL`.
- **XA\_API\_FATAL\_MEM\_ALIGN**  
One of the pointers (`handle` or `p_param_value`) is not aligned correctly.
- **XA\_OGG\_EXECUTE\_FATAL\_NOT\_INITIALIZED**  
The function is called before HiFi Ogg Parser is initialized.

## 5.4.4 HiFi Ogg Packer/Parser Parameters

This section describes the parameters that are supported by the `xa_ogg_pack_set_param/xa_ogg_parse_set_param` and `xa_ogg_pack_get_param/xa_ogg_parse_get_param` functions described in the previous section.

Table 5-13 and Table 5-14 contain the following columns:

- Parameter name: Parameter identifier (param\_id).
- Value type: A pointer (p\_param\_value) to a variable of this type is to be passed.
- RW: Indicates whether the parameter can be read (get) and/or written (set).
- Range: The range or supported values of the parameter value.
- Default: Default value of the parameter.
- Description: Brief description of the parameter.

Table 5-13 HiFi Ogg Packer Parameters

| Parameter Name                  | Value Type | RW  | Range / Supported Values  | Default | Description                                                              |
|---------------------------------|------------|-----|---------------------------|---------|--------------------------------------------------------------------------|
| XA_OGG_PACK_PARAM_FRAME_SIZE    | WORD 32    | W   | Opus Supported Frame Size | NA      | Frame size in samples.                                                   |
| XA_OGG_PACK_PARAM_END_OF_STREAM | WORD 32    | R/W | 0, 1                      | NA      | Gets value 1 if the end of the stream is detected in the stream, else 0. |

Table 5-14 HiFi Ogg Parser Parameters

| Parameter Name                   | Value Type | RW | Range / Supported Values | Default | Description                                                                                               |
|----------------------------------|------------|----|--------------------------|---------|-----------------------------------------------------------------------------------------------------------|
| XA_OGG_PARSE_PARAM_PAGE_GRANULE  | WORD 64    | R  | Any positive value       | NA      | Gets the granule position for the current Ogg page. The pointer to this variable must be 8 bytes aligned. |
| XA_OGG_PARSE_PARAM_END_OF_STREAM | WORD 32    | R  | 0, 1                     | NA      | Gets value 1 if the end of the stream is detected in the stream, else 0.                                  |

## 5.5 Metadata for Non-Ogg/Raw Decoder

Few metadata parameters are required for decoding raw multichannel encoded test streams. Some are fixed for the complete session, and some vary for each packet. The Ogg decoder does not require these, as this information is available in the Ogg header. Decoding raw encoded streams is seamless with the Cadence decoder. For other decoders, the following points should be considered.

The metadata parameters fixed for a complete decoding session are the number of channels, sample rate, stream count, coupled stream count, mapping family, and stream map.

In the Cadence decoder testbench, sample rate, number of channels, and mapping family are provided with command-line arguments. If the mapping family is not provided, set to default based on the number of channels. Other parameters, stream count, coupled stream count, and stream map, are derived at initialization from fixed mapping based on channel mapping family as per Opus standard (Ref: RFC7845 - Section 5.1.1.2).

The parameters that vary for every packet are sent with an encoded bitstream. Below is the bitstream structure of the raw encoded packet from the Cadence encoder. This structure is the same for all raw bitstreams, irrespective of the number of channels.

|                       |                            |                             |
|-----------------------|----------------------------|-----------------------------|
| <-- 4 bytes -->       | <-- 4 bytes -->            | <----- Packet Length -----> |
| <b>Payload length</b> | <b>Range Encoder state</b> | <b>Opus Encoded Packet</b>  |

- The first 4 bytes of the Payload length are used to communicate the payload size of that complete packet to the decoder.
- The range decoder state is used for conformance check at the decoder side (Ref: RFC6716 - Section 6). This check is not mandatory. On the Cadence decoder side, it can be disabled by setting the `no_range_dec_state` variable from the decoder control structure.

---

## 6. References

---

- [1] Definition of the Opus Audio Codec @ <http://tools.ietf.org/html/rfc6716>.  
Updates to the Opus Audio Codec draft-valin-codec-opus-update-00 @ <http://tools.ietf.org/html/draft-valin-codec-opus-update-00>
- [2] Reference Source Code (libopus 1.3.1) @ <http://opus-codec.org/downloads/>
- [3] The Ogg Encapsulation Format @ <https://www.xiph.org/ogg/doc/rfc3533.txt>
- [4] Ogg Encapsulation for Opus Audio Codec @ <https://tools.ietf.org/html/rfc7845>
- [5] [downloads.xiph.org/releases/ogg/libogg-1.3.2.tar.gz](http://downloads.xiph.org/releases/ogg/libogg-1.3.2.tar.gz)